BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

KHOA ĐIỆN ĐIỆN TỬ

BỘ MÔN KỸ THUẬT MÁY TÍNH



BÁO CÁO TIỂU LUẬN

ĐỀ TÀI

HỆ THỐNG SMARTLOCKER TÍCH HỢP PHẦN MỀM AI OCR VÀ IOT

GVHD: ThS. Huỳnh Hoàng Hà

SVTH:	MSSV
Nguyễn Thái Anh	23139002
Vũ Quốc Bảo	23139004
Nguyễn Võ Thái Sơn	23149040
Trần Ngọc Thiện	23139045
Đỗ Hoàng Xuân Tiến	23139047

TP. Hồ Chí Minh, tháng 10 năm 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn quý thầy cô đã trang bị kiến thức nền tảng và định hướng để nhóm thực hiện đề tài hệ thống SmartLocker tích hợp ứng dụng di động, AI OCR và phần cứng IoT.

Trong quá trình thực hiện, nhóm đã vận dụng kiến thức về phát triển ứng dụng, xây dựng Backend, cơ sở dữ liệu, nhận dạng ký tự và điều khiển thiết bị để hình thành một quy trình giao nhận kiện hàng thống nhất. Những góp ý trong quá trình học tập và hoàn thiện sản phẩm là cơ sở quan trọng để nhóm rà soát kiến trúc, chức năng và kết quả thực nghiệm.

Mặc dù đã cố gắng hoàn thiện, báo cáo khó tránh khỏi thiếu sót do giới hạn về thời gian, dữ liệu kiểm thử và phạm vi mô hình. Nhóm kính mong nhận được ý kiến góp ý để tiếp tục hoàn thiện hệ thống. Chúng em xin chân thành cảm ơn.

LỜI CAM ĐOAN

Chúng em xin cam đoan báo cáo về hệ thống SmartLocker tích hợp AI OCR và IoT được thực hiện trên cơ sở nghiên cứu, triển khai và kiểm thử trong phạm vi đề tài của nhóm.

Các nội dung mô tả kiến trúc, mã nguồn, hình ảnh, kết quả và giới hạn kỹ thuật được trình bày theo sản phẩm hiện có. Những tài liệu tham khảo được liệt kê ở cuối báo cáo. Nhóm chịu trách nhiệm về tính trung thực của nội dung trước khi nộp bài.

MỤC LỤC

LỜI CẢM ƠN i

LỜI CAM ĐOAN ii

MỤC LỤC iii

DANH MỤC BẢNG BIỂU v

DANH MỤC HÌNH ẢNH vi

GIỚI THIỆU 1

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI HỆ THỐNG SMARTLOCKER TÍCH HỢP PHẦN MỀM AI OCR VÀ IOT 2

1.1. Đặt vấn đề và lý do chọn đề tài 2

1.2. Mục tiêu của đề tài 2

1.3. Đối tượng và phạm vi thực hiện 3

1.4. Phương pháp thực hiện 4

1.5. Các chức năng chính của hệ thống 5

1.6. Cấu trúc báo cáo 6

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 8

2.1. Yêu cầu hệ thống 8

2.2. Tác nhân và Use Case 11

2.3. Kiến trúc tổng thể 13

2.4. Thiết kế luồng nghiệp vụ 15

2.5. Thiết kế Backend và cơ sở dữ liệu 18

CHƯƠNG 3. XÂY DỰNG PHẦN MỀM SMARTLOCKER 24

3.1. Công nghệ và môi trường phát triển 24

3.2. Kiến trúc Mobile App 25

3.3. Trang chủ, đăng ký và đăng nhập 30

3.4. Xây dựng chức năng Shipper 33

3.5. Xây dựng chức năng Cư dân 34

3.6. Xây dựng chức năng Quản trị viên và lịch sử 37

3.7. Cơ chế OTP và phân bổ ô tủ 42

CHƯƠNG 4. TÍCH HỢP AI OCR VÀ PHẦN CỨNG IOT 45

4.1. Vai trò của AI trong nghiệp vụ Shipper 45

4.2. Pipeline nhận dạng số điện thoại 45

4.3. Mô hình PP-OCRv6 và triển khai ONNX trên Android 48

4.4. Hậu xử lý và lựa chọn số điện thoại người nhận 50

4.5. Tích hợp AI với ShipperScreen 54

4.6. Thiết kế phần cứng ESP32 và relay 57

4.7. Ánh xạ ô tủ và GPIO 58

4.8. Firmware và giao tiếp với Backend 59

4.9. Tích hợp AI, phần mềm và IoT 63

CHƯƠNG 5. THỰC NGHIỆM, KẾT QUẢ VÀ ĐÁNH GIÁ 64

5.1. Môi trường và phạm vi kiểm thử 64

5.2. Kết quả kiểm thử tổng hợp 64

5.3. Đánh giá kết quả tích hợp 65

5.4. Hạn chế của hệ thống 65

5.5. Hướng phát triển 66

KẾT LUẬN 67

TÀI LIỆU THAM KHẢO 68

DANH MỤC BẢNG BIỂU

Bảng 1.1. Nhóm chức năng chính của hệ thống SmartLocker 5
Bảng 2.1. Yêu cầu chức năng và ràng buộc theo tác nhân 8
Bảng 2.2. Yêu cầu phi chức năng và giới hạn hiện tại 10
Bảng 2.3. Vai trò và ràng buộc của năm bảng SQLite 20
Bảng 3.1. Các công nghệ được sử dụng 24
Bảng 3.2. Vai trò của các nhóm tệp trong ứng dụng SmartLocker 27
Bảng 3.3. Một số nhóm giao tiếp giữa ứng dụng di động và Backend 29
Bảng 3.4. Thông tin đăng ký theo từng vai trò 30
Bảng 3.5. Phân quyền điều hướng sau đăng nhập 32
Bảng 3.6. Các chỉ số thống kê trên màn hình Admin 38
Bảng 3.7. Các loại sự kiện nghiệp vụ được ghi nhận trong lịch sử SmartLocker 42
Bảng 3.8. Điều kiện xác định khả năng phân bổ ô tủ 44
Bảng 4.1. Các bước tiền xử lý ảnh trong native module 47
Bảng 4.2. Các tệp mô hình và từ điển triển khai trên Android 48
Bảng 4.3. Cấu hình suy luận của Detector và Recognizer 49
Bảng 4.4. Quy tắc chấm điểm ứng viên SĐT 52
Bảng 4.5. Quy tắc quyết định kết quả trong một hướng xoay 53
Bảng 4.6. Các phương thức của SmartLockerOcrNativeModule 55
Bảng 4.7. Cấu hình hoạt động của firmware ESP32 57
Bảng 4.8. Ánh xạ ô tủ – kênh relay – GPIO 58
Bảng 4.9. Trạng thái của lệnh phần cứng 60
Bảng 4.10. Các khóa liên kết dữ liệu xuyên suốt hệ thống 63
Bảng 5.1. Tổng hợp kết quả kiểm thử các thành phần chính 64

DANH MỤC HÌNH ẢNH

Hình 1.1. Phạm vi và các thành phần của hệ thống SmartLocker.....4
Hình 1.2. Luồng hoạt động tổng quát của hệ thống SmartLocker 6
Hình 2.1. Sơ đồ Use Case theo tác nhân của hệ thống SmartLocker.....12
Hình 2.2. Sơ đồ kiến trúc tổng thể hệ thống SmartLocker.....14
Hình 2.3. Sơ đồ trình tự nghiệp vụ giao hàng 16
Hình 2.4. Sơ đồ trình tự nghiệp vụ nhận hàng ở bản hiện tại.....17
Hình 2.5. Sơ đồ kiến trúc Backend theo trách nhiệm xử lý 19
Hình 2.6. Sơ đồ quan hệ dữ liệu logic và khóa ngoại.....22
Hình 3.1. Sơ đồ điều hướng các màn hình theo vai trò người dùng.....28
Hình 3.2. Giao diện đăng ký tài khoản Shipper và Admin. 31
Hình 3.3. Bảng thống kê tổng quan tình trạng hoạt động của SmartLocker. 39
Hình 3.4. Giao diện giám sát trạng thái các ô tủ SmartLocker. 40
Hình 4.1. Ví dụ minh họa luồng OCR và trích xuất SĐT người nhận.....46
Hình 4.2. Bill chụp nghiêng (trái) và kết quả trên App Shipper khi phải chạy đủ bốn hướng xoay (phải).....54
Hình 4.3. Màn hình App Shipper khi chụp bill (trái) và hộp thoại kết quả nhận diện (phải).....56
Hình 4.4. Kết nối ESP32 với module relay 4 kênh 58
Hình 4.5. Sơ đồ giao tiếp Backend - ESP32 - relay 61
Hình 4.6. Mở tủ trên App Cư dân và trạng thái module relay: (a), (c) hộp thoại "Mở tủ thành công" của tủ B05 và A02; (b), (d) relay tương ứng được kích.....62

GIỚI THIỆU

Sự phát triển của thương mại điện tử làm gia tăng nhu cầu giao nhận kiện hàng tại chung cư, ký túc xá và văn phòng. SmartLocker giúp tách thời điểm giao hàng khỏi thời điểm nhận hàng, đồng thời yêu cầu hệ thống phải xác định đúng người nhận, quản lý trạng thái ô tủ và điều khiển phần cứng an toàn.

Đề tài xây dựng mô hình gồm ứng dụng React Native cho Shipper, Cư dân và Quản trị viên; Backend Node.js và Express quản lý nghiệp vụ; SQLite lưu dữ liệu; Firebase Cloud Messaging gửi thông báo; PP-OCRv6 chạy qua ONNX Runtime hỗ trợ đọc số điện thoại trên bill; ESP32 nhận lệnh và điều khiển relay.

Báo cáo trình bày lần lượt tổng quan đề tài, phân tích và thiết kế hệ thống, quá trình xây dựng phần mềm, tích hợp AI OCR và IoT, sau đó tổng hợp kết quả kiểm thử, hạn chế và hướng phát triển. Nội dung tập trung vào các quyết định thiết kế và bằng chứng chính, tránh lặp lại chi tiết triển khai ở nhiều chương.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI HỆ THỐNG SMARTLOCKER TÍCH HỢP PHẦN MỀM AI OCR VÀ IOT

1.1. Đặt vấn đề và lý do chọn đề tài

Sự phát triển của thương mại điện tử làm tăng nhanh số lượng kiện hàng được giao đến chung cư, ký túc xá, khu dân cư và văn phòng. Trong mô hình giao nhận trực tiếp, Shipper và người nhận phải có mặt cùng thời điểm. Khi người nhận vắng mặt, kiện hàng thường được gửi tại quầy bảo vệ hoặc khu vực chung, dẫn đến nguy cơ thất lạc, nhận nhầm và khó truy vết trách nhiệm. Việc liên lạc nhiều lần cũng kéo dài thời gian giao hàng và làm giảm hiệu quả vận hành.

Tủ nhận hàng thông minh có thể tách thời điểm giao hàng khỏi thời điểm nhận hàng. Tuy nhiên, một hệ thống hoàn chỉnh không chỉ cần cơ cấu mở khóa. Hệ thống còn phải xác định đúng người nhận, ngăn người không có quyền truy cập, tránh cấp trùng ô tủ, quản lý vòng đời kiện hàng và lưu bằng chứng lịch sử. Nếu ứng dụng điều khiển relay trực tiếp, trạng thái vật lý và dữ liệu nghiệp vụ dễ bị tách rời; nếu toàn bộ thông tin trên bill được nhập thủ công, thao tác của Shipper chậm và dễ sai số.

Từ yêu cầu đó, đề tài lựa chọn kiến trúc có Backend làm trung tâm. Mobile App cung cấp giao diện riêng cho Shipper, Cư dân và Quản trị viên; Backend xác thực, phân quyền, cấp ô tủ, quản lý OTP, lưu dữ liệu và tạo lệnh phần cứng; Firebase Cloud Messaging gửi thông báo tới Cư dân; ESP32 định kỳ nhận lệnh, điều khiển relay và phản hồi kết quả. AI OCR chạy cục bộ trên Android giúp nhận dạng số điện thoại người nhận từ ảnh bill trước khi Shipper xác nhận tạo kiện hàng.

Cách tổ chức này phù hợp với mục tiêu của một đồ án tích hợp: phần mềm chịu trách nhiệm điều phối và duy trì nguồn trạng thái đáng tin cậy; AI hỗ trợ giảm thao tác nhập liệu; IoT chuyển lệnh số thành tác động vật lý. Đồng thời, việc tách các lớp giúp dễ kiểm thử, bảo trì và mở rộng hơn so với một ứng dụng điều khiển phần cứng trực tiếp mà không có Backend trung tâm.

1.2. Mục tiêu của đề tài

Mục tiêu tổng quát: Xây dựng mô hình SmartLocker hỗ trợ giao và nhận kiện hàng qua ứng dụng di động, trong đó phần mềm điều phối toàn bộ quy trình; AI OCR hỗ trợ nhận dạng số điện thoại người nhận; ESP32 thực thi lệnh mở hoặc đóng relay theo chỉ đạo từ Backend.

Các mục tiêu cụ thể gồm:

- Xây dựng ứng dụng React Native cho ba vai trò Quản trị viên, Cư dân và Shipper; mỗi vai trò phải đăng nhập và chỉ truy cập đúng chức năng được cấp.
- Xây dựng REST API bằng Node.js và Express, lưu dữ liệu bằng SQLite và triển khai Backend trên Railway với volume để duy trì tệp cơ sở dữ liệu.
- Tạo luồng giao hàng từ số điện thoại người nhận đến phân bổ ô tủ, tạo package, ghi lịch sử và gửi thông báo Firebase Cloud Messaging.
- Tạo luồng nhận hàng bằng OTP có thời hạn, xác minh quyền sở hữu package và điều phối hardware command theo trạng thái.
- Tích hợp PP-OCRv6 chạy bằng ONNX Runtime trên Android để nhận dạng số điện thoại từ ảnh bill và trả kết quả về màn hình Shipper.
- Xây dựng firmware ESP32 có khả năng xác thực với Backend, lấy lệnh OPEN hoặc CLOSE, điều khiển bốn kênh relay và gửi ACK kết quả.
- Đánh giá khách quan khả năng hoạt động, giới hạn của prototype và các hạng mục cần hoàn thiện trước khi triển khai thực tế.

1.3. Đối tượng và phạm vi thực hiện

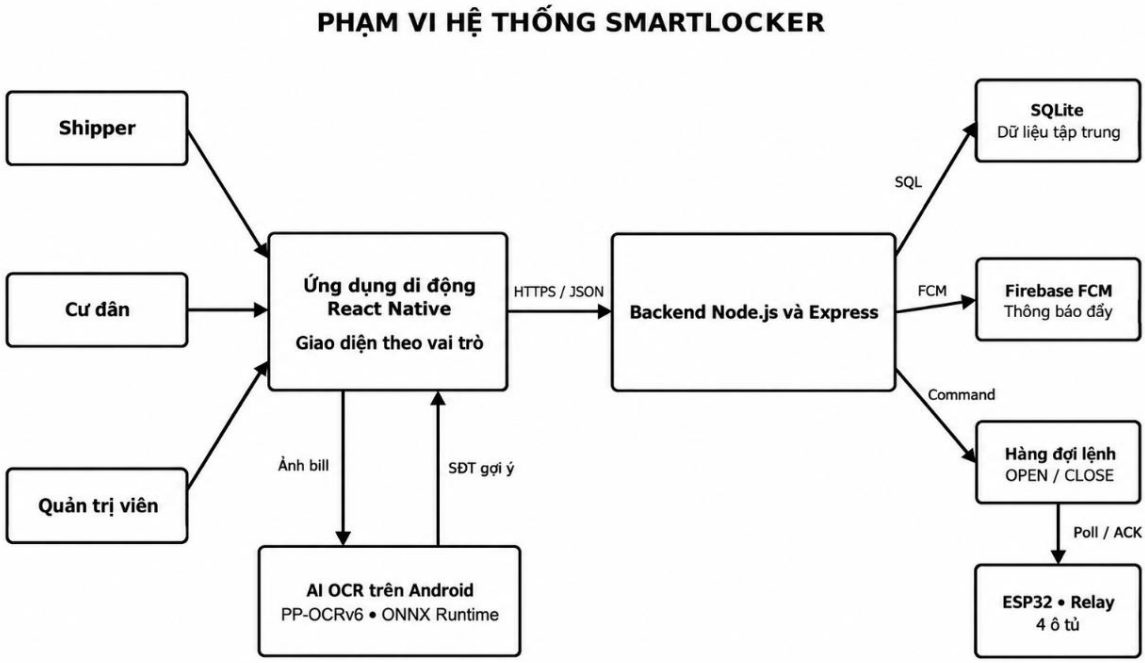
Đối tượng thực hiện: Đối tượng của đề tài là một hệ thống SmartLocker tích hợp ở quy mô mô hình, gồm ứng dụng Android, Backend triển khai trên Railway, cơ sở dữ liệu SQLite, dịch vụ thông báo FCM, mô-đun OCR trên thiết bị và bộ điều khiển ESP32 nối với bốn kênh relay.

Phạm vi phần mềm: Hệ thống quản lý ba vai trò người dùng, phiên đăng nhập bằng JWT, package, OTP, thiết bị nhận thông báo, lịch sử hoạt động và hàng đợi lệnh phần cứng. Bốn ô tủ được xác định trong mã nguồn hiện tại là A01, A02, B01 và B05. Backend hỗ trợ chế độ SOFTWARE để mô phỏng và chế độ ESP32 để điều khiển thiết bị thật.

Phạm vi AI: AI chỉ giải quyết bài toán trích xuất số điện thoại người nhận trên bill giao hàng. Hệ thống không nhận dạng danh tính sinh trắc học, không phân loại toàn bộ nội dung vận đơn và không tự động tạo package khi Shipper chưa kiểm tra kết quả.

Phạm vi phần cứng: Firmware chịu trách nhiệm kết nối Wi-Fi, gọi API HTTPS, nhận lệnh, ánh xạ locker sang GPIO, điều khiển relay và ACK kết quả. Đề tài không đi

sâu vào thiết kế cơ khí tủ, lựa chọn loại khóa điện, tính toán nguồn công suất hoặc thiết kế mạch bảo vệ ngoài những gì được thể hiện trong mô hình và firmware hiện có.



Hình 1.1. Phạm vi và các thành phần của hệ thống SmartLocker

Hình 1.1 thể hiện trách nhiệm giữa các thành phần. Người dùng chỉ tương tác qua Mobile App. Backend là điểm kiểm soát quyền và trạng thái; SQLite lưu dữ liệu nghiệp vụ; FCM vận chuyển thông báo; AI OCR hỗ trợ Shipper; ESP32 nhận command từ Backend thay vì nhận lệnh trực tiếp từ điện thoại.

1.4. Phương pháp thực hiện

Đề tài được triển khai theo phương pháp phân tích yêu cầu, thiết kế kiến trúc, phát triển theo mô-đun và tích hợp tăng dần. Cách tiếp cận này giúp mỗi lớp có thể được kiểm tra riêng trước khi nối thành một quy trình hoàn chỉnh.

1. Phân tích nghiệp vụ theo vai trò, xác định quyền của Shipper, Cư dân, Quản trị viên và thiết bị ESP32; đồng thời xác định trạng thái của package và hardware command.
2. Thiết kế dữ liệu và API làm nền tảng trung tâm, trong đó SQLite lưu trạng thái còn Backend kiểm tra JWT, role, OTP và device key.
3. Phát triển Mobile App theo từng màn hình và service, sau đó kết nối tới API, AsyncStorage, camera và Firebase Messaging.

4. Tích hợp AI dưới dạng native module Android để thực hiện tiền xử lý ảnh, suy luận detector và recognizer, hậu xử lý số điện thoại rồi trả kết quả về ShipperScreen.
5. Thống nhất giao thức OPEN, CLOSE, claim và ACK giữa Backend với ESP32; ánh xạ từng locker tới kênh relay và GPIO tương ứng.
6. Đánh giá bằng kiểm tra tĩnh mã nguồn, đối chiếu schema và route, xác minh checksum model, sử dụng bằng chứng benchmark có sẵn và chuẩn bị các kịch bản chạy thực tế trên điện thoại, Railway và ESP32.

1.5. Các chức năng chính của hệ thống

Các chức năng được tổ chức theo vai trò và lớp kỹ thuật. Việc phân tách này giúp giao diện dễ sử dụng, đồng thời ngăn một vai trò gọi chức năng không thuộc quyền của mình. Bảng 1.1 tóm tắt trách nhiệm chính của từng nhóm.

Bảng 1.1. Nhóm chức năng chính của hệ thống SmartLocker

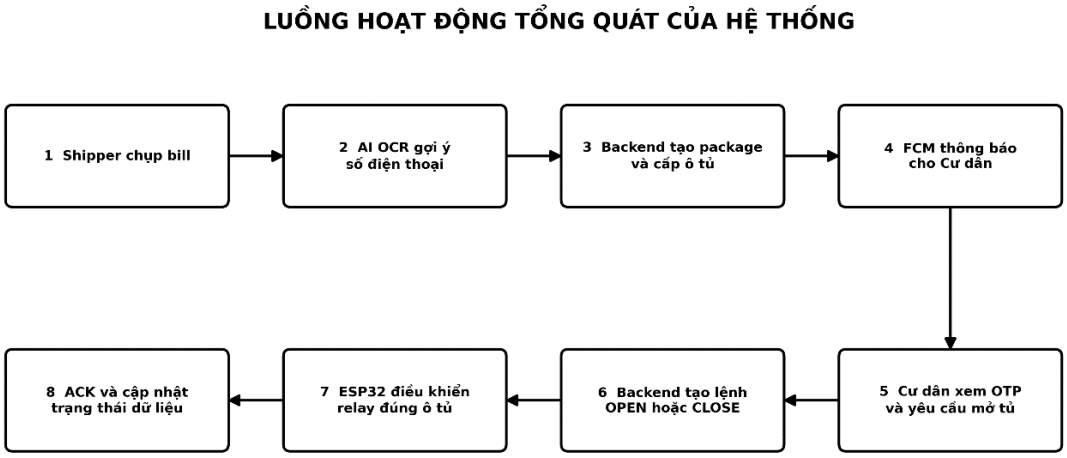
Nhóm	Chức năng chính
Xác thực	Đăng ký và đăng nhập bằng số điện thoại, mật khẩu và vai trò; Admin và Shipper cần mã đăng ký tương ứng.
Shipper	Chụp bill, chạy OCR, kiểm tra hoặc chỉnh số điện thoại, tạo package và nhận ô tủ do Backend cấp.
Cư dân	Đăng ký token FCM, nhận thông báo, xem package, xem hoặc cấp lại OTP và gửi yêu cầu mở tủ.
Quản trị viên	Xem thống kê và trạng thái bốn ô, xem lịch sử, mở khẩn cấp và theo dõi phản hồi ESP32.
Backend	Xác thực, phân quyền, quản lý package, cấp tủ, OTP, FCM, lịch sử và hardware command.
ESP32	Kết nối Wi-Fi và HTTPS, lấy lệnh, điều khiển relay theo locker và ACK kết quả.

Luồng giao hàng: Shipper đăng nhập, chụp bill và sử dụng OCR để lấy gợi ý số điện thoại. Sau khi xác nhận, ứng dụng gửi yêu cầu tạo package. Backend tìm tài khoản Cư dân, cấp ô đang trống, sinh OTP, lưu package trạng thái READY, ghi lịch sử và gửi FCM.

Luồng nhận hàng: Cư dân nhận thông báo, mở danh sách package và dùng OTP còn hiệu lực để yêu cầu mở tủ. Backend xác minh quyền sở hữu và tạo command. ESP32

lấy lệnh, điều khiển relay theo locker rồi gửi ACK; ứng dụng theo dõi trạng thái command để phản hồi cho người dùng.

Luồng quản trị: Quản trị viên xem trạng thái bốn ô, thống kê sử dụng, lịch sử giao dịch và có thể mở khẩn cấp package READY. Các thao tác quản trị vẫn phải đi qua Backend và được ghi nhận vào dữ liệu tập trung.



Hình 1.2. Luồng hoạt động tổng quát của hệ thống SmartLocker

Luồng tổng quát cho thấy AI xuất hiện ở đầu quy trình để hỗ trợ nhập dữ liệu, còn IoT xuất hiện ở cuối quy trình để thực thi lệnh vật lý. Phần mềm kết nối hai đầu này bằng dữ liệu package, locker, OTP, notification và hardware command. Nhờ đó, một thao tác mở tủ luôn có ngữ cảnh nghiệp vụ và có thể truy vết.

1.6. Cấu trúc báo cáo

Báo cáo được tổ chức thành năm chương.

Chương 1 trình bày bối cảnh, mục tiêu, phạm vi, phương pháp và chức năng chính.

Chương 2 phân tích yêu cầu, tác nhân, kiến trúc, luồng nghiệp vụ, cơ sở dữ liệu và API.

Chương 3 trình bày chi tiết việc xây dựng Mobile App, Backend, SQLite, FCM, OTP và cơ chế điều phối hardware command.

Chương 4 mô tả pipeline AI OCR và phần cứng ESP32 trong mối quan hệ với phần mềm.

Chương 5 trình bày môi trường kiểm thử, bằng chứng hiện có, kết quả đánh giá, ưu điểm, hạn chế và rủi ro vận hành. Phần cuối gồm kết luận, hướng phát triển, tài liệu tham khảo.

Với cấu trúc này, phần mềm được trình bày như trục chính của hệ thống; AI và IoT được phân tích theo vai trò tích hợp, tránh tách rời khỏi quy trình giao nhận.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Hệ thống SmartLocker được thiết kế để quản lý giao nhận kiện hàng theo ba vai trò Shipper, Cư dân và Quản trị viên. Backend giữ dữ liệu nghiệp vụ và quyết định quyền truy cập, ứng dụng Android cung cấp giao diện, nhận thông báo và hỗ trợ đọc bill bằng AI OCR, ESP32 thực hiện lệnh điều khiển relay. Chương này xác định các yêu cầu, tác nhân, kiến trúc, luồng nghiệp vụ, mô hình dữ liệu, giao diện API và cơ chế bảo vệ trạng thái để làm cơ sở triển khai phần mềm ở Chương 3 và tích hợp thiết bị ở Chương 4.

Phạm vi thiết kế tương ứng với mô hình bốn ô A01, A02, B01 và B05. Trong cấu hình cấp tủ, cả bốn ô đều có kích thước quy ước M, đây là thuộc tính phục vụ phần mềm demo, không phải số đo cơ khí. Hệ thống có hai chế độ SOFTWARE và ESP32. SOFTWARE mô phỏng hoàn tất nhận hàng tại Backend, ESP32 tách thao tác mở relay và đóng relay thành hai lệnh có xác nhận. Việc phân biệt hai chế độ là cần thiết vì kết quả mô phỏng không chứng minh rằng cơ cấu tủ thật đã hoạt động.

2.1. Yêu cầu hệ thống

2.1.1. Yêu cầu chức năng

Yêu cầu chức năng được hình thành từ các thao tác trên ứng dụng và các điều kiện kiểm tra trong API. Một giao dịch hợp lệ phải xác định được người gửi, người nhận, ô tủ, trạng thái kiện hàng và sự kiện tương ứng trong lịch sử. Các quyền nhạy cảm phải được kiểm tra tại Backend ngay cả khi giao diện đã giới hạn màn hình theo vai trò.

Bảng 2.1. Yêu cầu chức năng và ràng buộc theo tác nhân

Mã	Nhóm	Chức năng	Ràng buộc
FR01	Tài khoản	Đăng ký, đăng nhập và kiểm tra phiên theo phone, password, role.	Admin/Shipper cần mã đăng ký, JWT cho API bảo vệ.
FR02	Shipper	Chụp bill, OCR cục bộ, chỉnh sửa hoặc nhập số điện thoại.	Kết quả AI phải được người dùng xác nhận.
FR03	Shipper	Tạo package, cấp ô phù hợp, ghi GỬI HÀNG, gửi FCM.	Chỉ SHIPPER, người nhận phải là RESIDENT, không trả OTP.
FR04	Cư dân	Đăng ký FCM token và xem kiện thuộc tài khoản.	Đối chiếu receiver_user_id với userId.

Mã	Nhóm	Chức năng	Ràng buộc
FR05	Cư dân	Xem OTP còn hạn, cấp lại và yêu cầu mở bằng OTP.	Kiện READY, OTP đúng và chưa hết hạn.
FR06	Nhận hàng	Xếp OPEN, sau OPEN DONE, xếp CLOSE để hoàn tất.	Backend/firmware đã có CLOSE, ResidentScreen chưa gọi.
FR07	Admin	Xem tổng quan, lịch sử và mở khẩn cấp.	ADMIN, package READY, ghi sự kiện can thiệp.
FR08	ESP32	Xác thực, claim lệnh, điều khiển relay và ACK.	Device key riêng, command OPEN/CLOSE.
FR09	Backend	Theo dõi command, phục hồi claim quá hạn và chống log trùng.	Một active command/package, event_key duy nhất.

Đăng ký và đăng nhập sử dụng số điện thoại, mật khẩu và vai trò. Backend loại bỏ ký tự không phải chữ số và kiểm tra số điện thoại có mười chữ số, bắt đầu bằng 0, quy tắc này kiểm tra định dạng, chưa xác minh quyền sở hữu số điện thoại qua SMS. Mật khẩu đăng ký có ít nhất sáu ký tự. Cư dân được đăng ký trực tiếp, còn tài khoản Admin và Shipper phải cung cấp mã đăng ký đúng với cấu hình. Cặp phone và role là duy nhất, vì vậy một số điện thoại có thể tồn tại ở nhiều vai trò nhưng không được tạo trùng trong cùng vai trò.

Đối với giao hàng, Shipper có thể chụp bill để nhận gợi ý số điện thoại hoặc nhập thủ công. OCR không phải điều kiện bắt buộc để tạo kiện hàng, dữ liệu cuối cùng do Shipper kiểm tra và xác nhận. Backend chỉ tiếp nhận người nhận đã có tài khoản RESIDENT, tự chọn ô còn trống và tạo package READY. Shipper nhận mã kiện và ô tủ, không nhận OTP lấy hàng. Route tạo package hiện chưa tạo lệnh OPEN dành cho thao tác bỏ hàng vào tủ, vì vậy bước đặt hàng thực tế vẫn cần được tổ chức trong quy trình demo.

Đối với nhận hàng, Cư dân xem danh sách kiện thuộc tài khoản, xem OTP còn hiệu lực, cấp lại OTP và gửi yêu cầu nhận hàng. Trong ESP32 mode, OTP hợp lệ dẫn đến việc xếp lệnh OPEN, kiện vẫn READY sau khi relay được bật. Chỉ khi lệnh CLOSE được xác nhận thành công, Backend mới chuyển kiện sang PICKED_UP và giải phóng ô tủ theo trạng thái dữ liệu. Backend đã cung cấp API CLOSE, nhưng ResidentScreen

hiện chưa gọi API này, đây là khoảng cách giữa luồng thiết kế đầy đủ và tích hợp phía Cư dân.

Các quy tắc nghiệp vụ cần duy trì gồm: một package có một người nhận, một ô đang chứa package READY không được cấp lại, OTP phải gắn với đúng package và đúng chủ tài khoản, một package chỉ có tối đa một command đang PENDING hoặc CLAIMED, ACK mở relay không tương đương với hoàn tất nhận hàng. CANCELLED có trong miền trạng thái của cơ sở dữ liệu nhưng chưa có endpoint hủy kiện ở phiên bản hiện tại.

2.1.2. Yêu cầu phi chức năng

Yêu cầu phi chức năng tập trung vào bảo mật, tính nhất quán, khả năng phục hồi và khả năng bảo trì của mô hình. Các thông số timeout trong mã nguồn là giới hạn chờ của phần mềm, chúng không phải kết quả đo độ trễ, độ chính xác OCR hoặc mức sẵn sàng của dịch vụ.

Bảng 2.2. Yêu cầu phi chức năng và giới hạn hiện tại

Thuộc tính	Cơ chế thiết kế	Giới hạn cần xét
Bảo mật	Bcrypt, JWT, kiểm tra role/ownership, OTP bảo vệ và có hạn.	Chưa có rate limit, ESP32 chưa xác minh chứng chỉ.
Nhất quán	Dữ liệu chính tại Backend, transaction claim/finalize, unique active command.	Chưa có ràng buộc một READY/locker ở schema.
Phục hồi	Lưu command trong SQLite, claim quá hạn được trả lại hàng đợi.	App timeout không hủy command, cần đối soát ACK muộn.
Khả dụng	Giao diện tải, thông báo lỗi, refresh và nhập tay nếu OCR không đạt.	Chưa có số đo SLA hoặc thử tải để khẳng định độ sẵn sàng.
Hiệu năng	Resize ảnh, inference native, prepared SQL và index.	Timeout 12 giây cho auth, chờ command 45 giây là cấu hình.
Bảo trì	Tách màn hình/service/config, schema và OCR native có module riêng.	Backend phần lớn còn trong server.js.

Thuộc tính	Cơ chế thiết kế	Giới hạn cần xét
Lưu bền vững	DB_PATH hoặc volume, WAL và khóa ngoại được bật.	Cần cấu hình volume và quy trình sao lưu thực tế.
Riêng tư	Không trả OTP cho Shipper, OCR cục bộ, giới hạn dữ liệu theo chủ.	Token AsyncStorage cần được bảo vệ tốt hơn khi vận hành.

Tính nhất quán được hiểu ở hai cấp. Ở cấp nghiệp vụ, Backend quyết định package nào đang chiếm tủ và tài khoản nào được nhận hàng. Ở cấp thiết bị, hardware_commands ghi lại yêu cầu, lần claim và kết quả ACK. Hai cấp này được liên kết nhưng không đồng nhất: dữ liệu READY có thể tồn tại sau OPEN DONE, còn relay thực tế có thể thay đổi khi ESP32 khởi động lại. Do chưa có cảm biến cửa hoặc cảm biến hiện diện kiện hàng trong luồng xác nhận, hệ thống chỉ biết kết quả điều khiển mà firmware báo về.

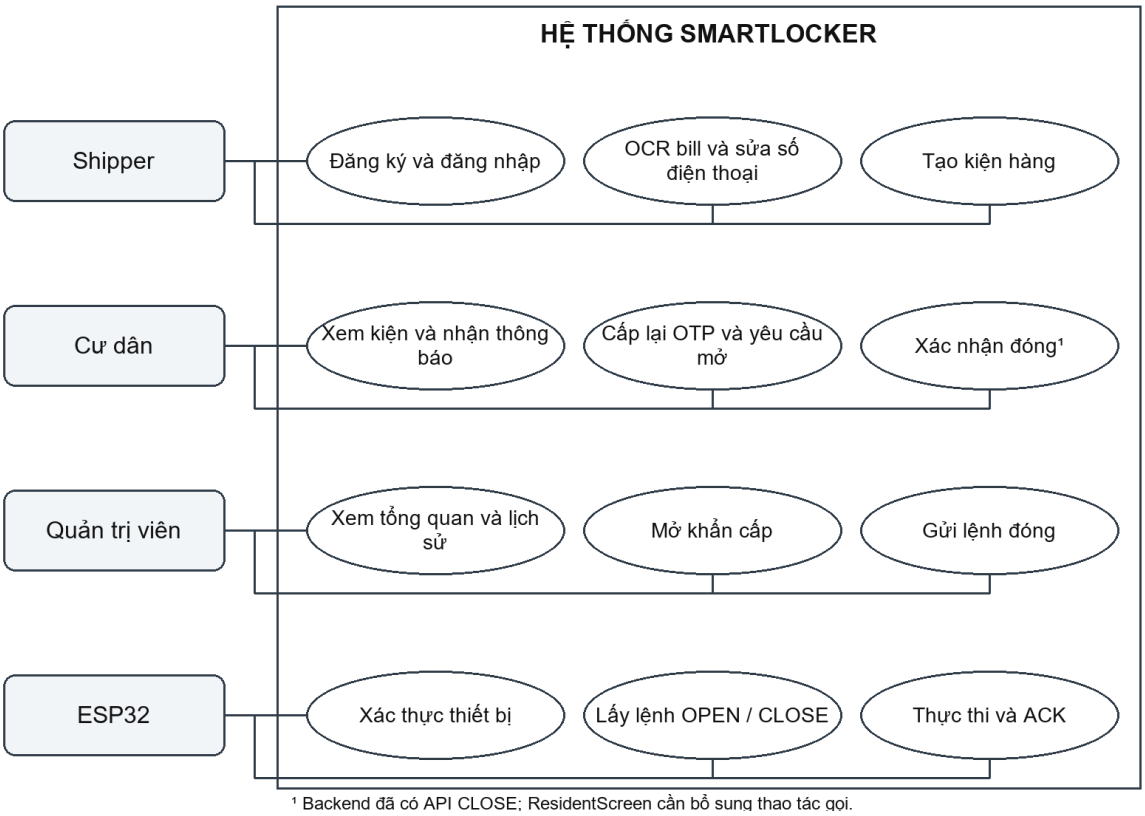
Khả năng bảo trì đến từ phân chia màn hình, context, service và cấu hình trong ứng dụng, database.js chịu trách nhiệm schema và migration, server.js giữ route và nghiệp vụ, module OCR native được đóng gói riêng. Đây là sự phân chia trách nhiệm trong prototype. Phần lớn logic Backend vẫn nằm trong một file, vì vậy khi mở rộng cần tách router, service và lớp truy cập dữ liệu để dễ kiểm thử và thay đổi.

2.2. Tác nhân và Use Case

2.2.1. Xác định tác nhân và phạm vi quyền

Shipper là tác nhân khởi tạo giao dịch giao hàng. Tác nhân này đăng nhập bằng role SHIPPER, sử dụng camera/OCR, chỉnh số điện thoại và gửi yêu cầu tạo package. Shipper không có quyền xem OTP, nhận kiện thay Cư dân hoặc gọi API quản trị. Danh tính người gửi được Backend tạo từ số điện thoại trong JWT, thay vì tin vào trường sender tự khai báo từ client.

Cư dân là chủ sở hữu kiện hàng thông qua packages.receiver_user_id. Tác nhân này đăng ký thiết bị nhận FCM, xem kiện của mình, lấy hoặc cấp lại OTP và yêu cầu mở tủ. Quyền truy cập không chỉ dựa trên việc biết public_id: Backend phải đối chiếu userId trong token với receiver_user_id. Khi chuyển tài khoản trên ứng dụng, thông báo chỉ được mở trực tiếp nếu vai trò và số điện thoại phù hợp với người nhận.



Hình 2.1. Sơ đồ Use Case theo tác nhân của hệ thống SmartLocker

Quản trị viên theo dõi tổng quan bốn ô, trạng thái kiện và lịch sử giao dịch. Trong tình huống cần can thiệp, Admin tạo lệnh mở khẩn cấp mà không cần OTP của Cư dân, ở ESP32 mode vẫn phải chờ ACK và dùng luồng CLOSE để hoàn tất. Quyền này có phạm vi rộng nên phải được kiểm tra bằng role ADMIN trên API và ghi sự kiện MỞ KHẨN CẤP khi giao dịch hoàn tất.

ESP32 là tác nhân thiết bị, không phải tài khoản người dùng. Thiết bị xác thực bằng X-Device-Key, gửi X-Device-Id, lấy lệnh kế tiếp và báo kết quả. Firebase Cloud Messaging là dịch vụ bên ngoài hỗ trợ chuyển thông báo đến thiết bị Android. AI OCR chạy cục bộ và được xem là thành phần bên trong ứng dụng, không phải một người dùng có quyền tạo giao dịch độc lập.

2.2.2. Danh mục và quan hệ Use Case

Nhóm Use Case chung gồm UC01 Đăng ký tài khoản và UC02 Đăng nhập hoặc khôi phục phiên đã nhớ. Nhóm Shipper gồm UC03 Nhận dạng số điện thoại trên bill và UC04 Tạo kiện hàng. Nhóm Cư dân gồm UC05 Xem kiện và nhận thông báo, UC06 Cấp lại OTP, UC07 Yêu cầu mở tủ và UC08 Xác nhận đóng để hoàn tất nhận hàng.

Nhóm Admin gồm UC09 Xem tổng quan và lịch sử, UC10 Mở khẩn cấp. Tác nhân ESP32 tham gia UC11 Nhận lệnh và ACK.

UC04 yêu cầu kết quả số điện thoại đã được xác nhận nhưng không bắt buộc thực hiện UC03: nhập tay là một nhánh hợp lệ. UC07 bao gồm xác minh phiên, quyền sở hữu, trạng thái kiện, thời hạn và giá trị OTP. UC06 được kích hoạt khi người dùng cần mã mới, thường sau khi mã cũ hết hạn, và không tự động thực thi mở tủ. UC08 phụ thuộc vào một OPEN đã DONE mà chưa được đóng, chức năng đã có phía Backend nhưng cần hoàn thiện kết nối từ ResidentScreen. UC10 thay bước kiểm tra OTP bằng quyền Admin, còn việc nhận lệnh và ACK vẫn theo UC11.

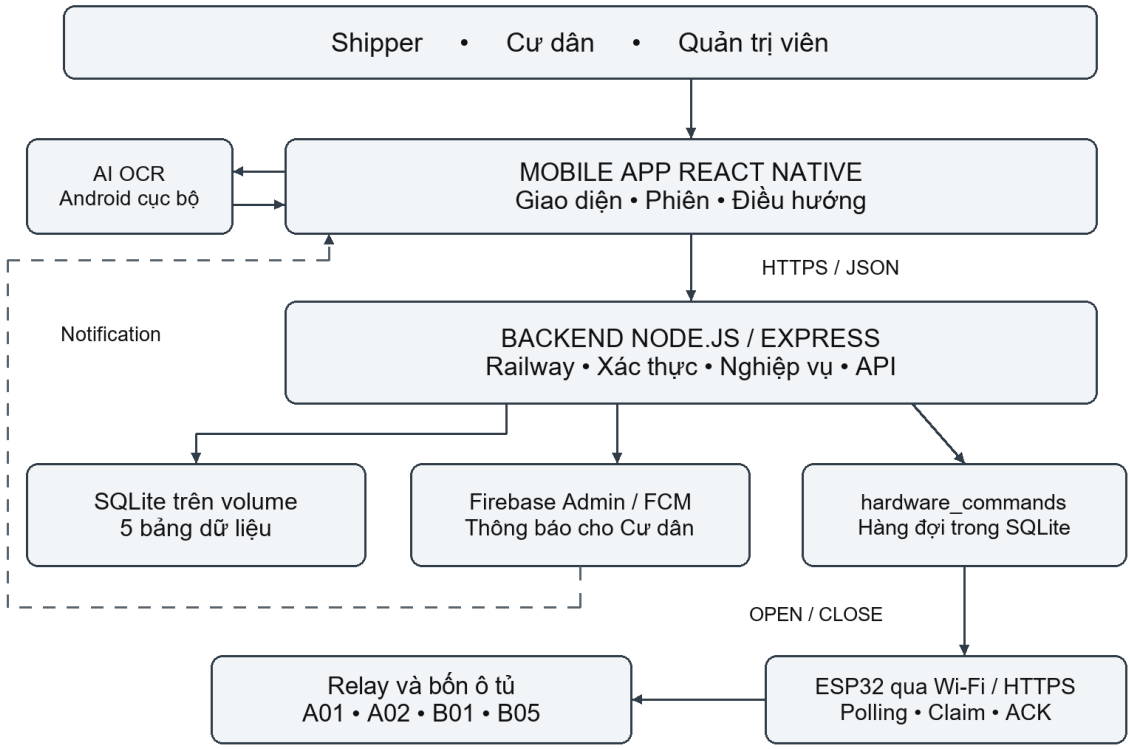
2.3. Kiến trúc tổng thể

2.3.1. Các thành phần và hướng trao đổi dữ liệu

Kiến trúc gồm lớp tương tác, lớp dịch vụ nghiệp vụ và lớp chấp hành. Lớp tương tác là Mobile App React Native trên Android với các màn hình Home, Login, Register, Shipper, Resident, Admin và History. AuthContext quản lý tài khoản và phiên, navigation quyết định màn hình theo vai trò và xử lý thông báo được mở. Ứng dụng gọi REST API bằng HTTPS và nhận JSON, nó không trực tiếp truy vấn SQLite hoặc phát tín hiệu GPIO.

Lớp dịch vụ là Backend Node.js/Express triển khai theo cấu hình Railway. Backend xác thực JWT, kiểm tra role và quyền sở hữu, phân bổ tủ, quản lý OTP, gửi FCM và điều phối command. SQLite qua better-sqlite3 giữ dữ liệu tập trung, đường dẫn cơ sở dữ liệu ưu tiên DB_PATH hoặc vị trí volume được cấu hình để bảo toàn dữ liệu qua lần triển khai lại. Firebase Admin là thành phần phía server gửi thông báo đến các FCM token đã đăng ký.

Lớp chấp hành gồm ESP32, bốn kênh relay và bốn ô tủ. ESP32 chủ động polling Backend qua Wi-Fi/HTTPS, Backend không yêu cầu mở kết nối trực tiếp vào mạng nội bộ của thiết bị. Command được lưu trong SQLite nên có thể tồn tại trong lúc ESP32 tạm mất kết nối. OPEN làm relay ON và giữ trạng thái, CLOSE làm relay OFF. Ánh xạ hiện tại là A01–GPIO25, A02–GPIO26, B01–GPIO27, B05–GPIO32, firmware dùng HIGH cho ON và LOW cho OFF.



Hình 2.2. Sơ đồ kiến trúc tổng thể hệ thống SmartLocker

2.3.2. Vai trò của AI OCR và dịch vụ thông báo

AI OCR nằm trong Android của Shipper. Pipeline sử dụng PP-OCRv6 detector, recognizer ONNX TopK-F32, giải mã CTC và hậu xử lý chọn số người nhận dựa trên ứng viên cùng ngữ cảnh người gửi/người nhận. Kotlin thực hiện phần xử lý ảnh và inference, smartlockerOcr.ts nhận kết quả và lựa chọn số điện thoại. Kiến trúc này không yêu cầu truyền ảnh bill đến Backend để nhận dạng. Dữ liệu được gửi khi tạo package là số điện thoại đã kiểm tra và size, vì vậy kết quả OCR không được phép tự động vượt qua xác nhận của người dùng.

FCM mang thông tin điều hướng như packageId, receiverPhone và locker. Thông báo giúp người nhận biết có kiện mới, nhưng không phải nguồn dữ liệu xác thực trạng thái và không thay thế kiểm tra quyền trên API. Khi mở notification, AppNavigator chỉ điều hướng thẳng đến ResidentScreen nếu tài khoản hiện tại phù hợp, nếu chưa đăng nhập hoặc đang dùng tài khoản khác, thông tin được giữ tạm trong AsyncStorage và người dùng quay về Home để chọn đúng tài khoản. ResidentScreen tiếp tục đọc package từ Backend.

2.3.3. Cơ sở lựa chọn kiến trúc

Backend tập trung giúp các vai trò sử dụng cùng quy tắc cấp tủ, cùng trạng thái kiện và cùng lịch sử. Hàng đợi command tách thời điểm gửi yêu cầu khỏi thời điểm thiết bị thực hiện, phù hợp với mạng Wi-Fi có thể gián đoạn. Polling đơn giản hóa firmware và triển khai mô hình, đổi lại, độ trễ phụ thuộc chu kỳ hỏi lệnh, thời gian mạng và quá trình ACK. Hệ thống hiện chưa dùng MQTT hoặc WebSocket để đẩy lệnh trực tiếp.

SQLite phù hợp với quy mô prototype vì cấu hình gọn và các thao tác dữ liệu chủ yếu diễn ra tại một dịch vụ Backend. WAL và index hỗ trợ đọc/ghi và truy vấn thường gặp. Khi cần nhiều Backend cùng ghi, nhiều cụm tủ hoặc lưu lượng lớn hơn, cần đánh giá lại cơ chế khóa, cấp tủ, định tuyến thiết bị và lựa chọn cơ sở dữ liệu. ALL_LOCKERS hiện là danh sách trong server.js, chưa có bảng quản lý cụm tủ hay ánh xạ tủ với nhiều thiết bị riêng biệt.

2.4. Thiết kế luồng nghiệp vụ

2.4.1. Luồng giao hàng

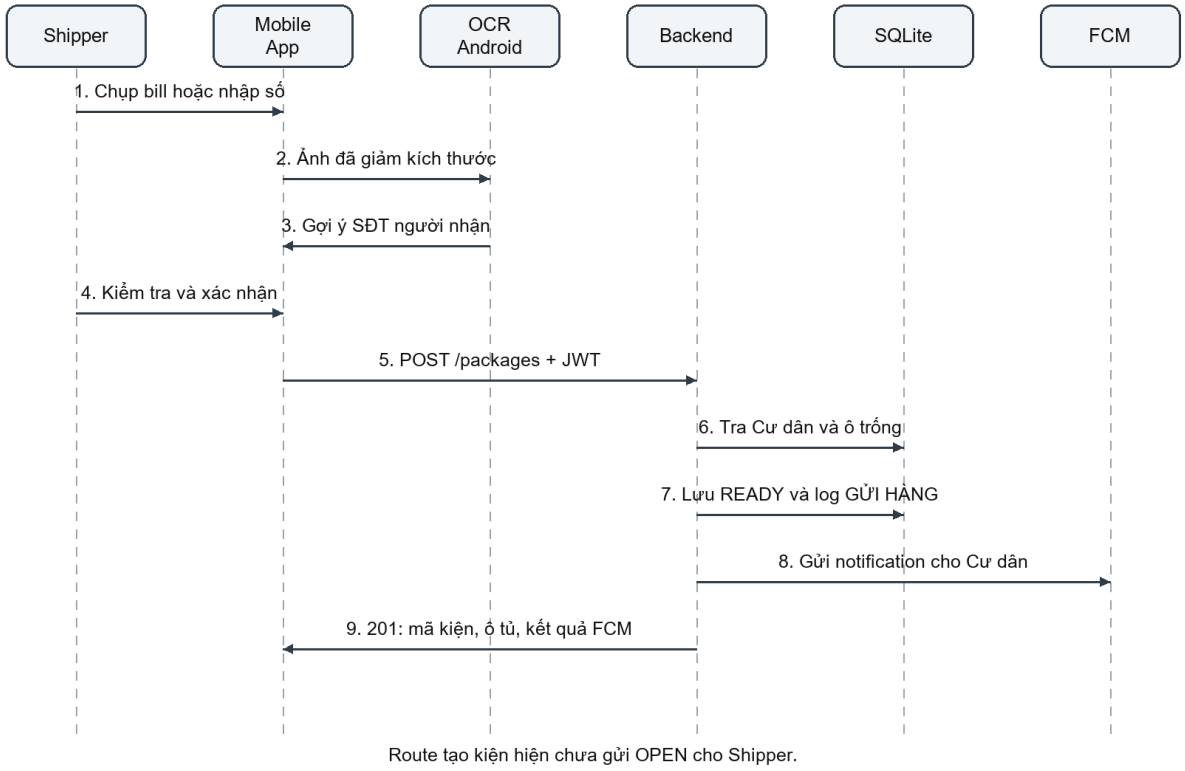
Luồng giao hàng bắt đầu khi Shipper đăng nhập đúng vai trò. Người dùng chụp bill nếu muốn dùng OCR, ứng dụng giảm kích thước ảnh và gọi native module để lấy văn bản cùng các ứng viên số điện thoại. Hậu xử lý lựa chọn ứng viên người nhận, sau đó hiển thị để người dùng chỉnh sửa. Nếu OCR không cho kết quả, Shipper nhập số điện thoại thủ công và vẫn đi tiếp bằng cùng một API tạo kiện.

Khi xác nhận, ShipperScreen gửi receiverPhone và size M trong POST /packages kèm Bearer token. Backend tạo sender từ danh tính đã xác thực, chuẩn hóa số và tìm tài khoản RESIDENT theo phone. Không tìm thấy thì dừng luồng với 404, sai định dạng thì trả 400. Backend không tự tạo tài khoản Cư dân khi nhận một số điện thoại mới.

Thuật toán cấp ô lấy tập locker của các package READY làm tập đã chiếm. Từ ALL_LOCKERS, hệ thống loại các ô bị chiếm và các ô có thứ hạng kích thước thấp hơn kiện, với S = 1, M = 2, L = 3. Các ô hợp lệ được sắp theo độ dư kích thước tăng dần, rồi theo mã ô. Do tất cả ô hiện là M, kiện S hoặc M có thể được cấp ô, kiện L không có ô phù hợp. Với các ô cùng kích thước và chưa bị chiếm, thứ tự lựa chọn là A01, A02, B01, B05.

Sau khi cấp ô, Backend tạo public_id dạng PKG_ kèm chuỗi định danh, tạo pickup_token phục vụ tương thích cũ, tạo bộ OTP và thời hạn. Package được lưu

READY, nhật ký GỬI HÀNG có event_key PACKAGE_CREATED gắn với public_id. Backend tìm các FCM token của người nhận, gửi thông báo và trả HTTP 201 với package cùng kết quả notification. Các trường hash, bản mã và OTP không xuất hiện trong phản hồi dành cho Shipper.



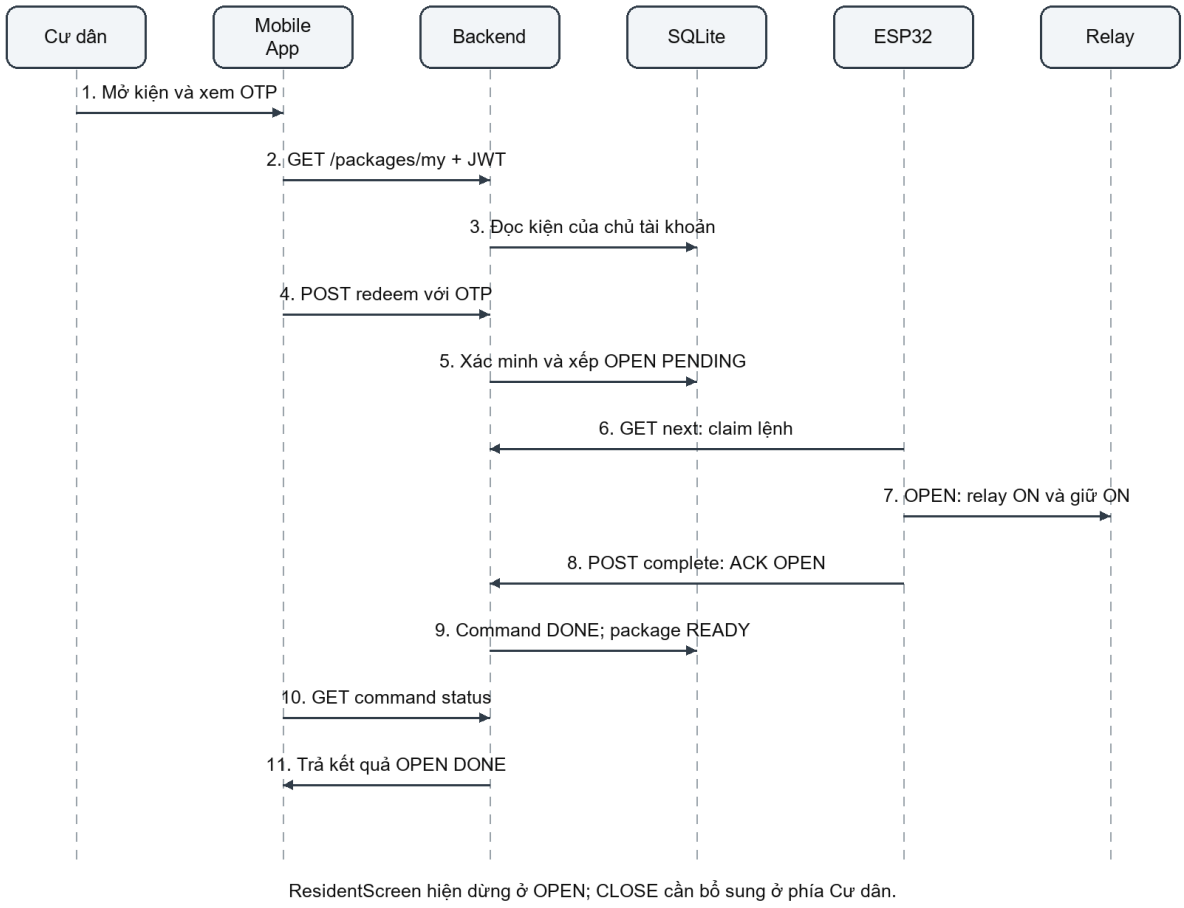
Hình 2.3. Sơ đồ trình tự nghiệp vụ giao hàng

Việc gửi FCM diễn ra sau khi package được ghi. Nếu không có token hoặc dịch vụ thông báo chưa sẵn sàng, người nhận vẫn có thể xem kiện bằng GET /packages/my khi mở ứng dụng. Không nên tạo lại kiện chỉ vì thông báo chưa đến, vì POST /packages chưa có khóa idempotency cho yêu cầu tạo đơn. Nếu client mất phản hồi sau khi Backend đã ghi dữ liệu, việc gửi lại có thể tạo một package khác, cần tra cứu trạng thái trước khi lặp thao tác trong vận hành.

Ở phiên bản hiện tại, route tạo package không xếp command mở tủ cho Shipper và không có cảm biến xác nhận hàng đã đặt vào ô. READY được tạo theo xác nhận phần mềm, vì vậy luồng giao hàng ở đây mô tả việc đăng ký kiện và giữ chỗ tủ. Để tự động hóa thao tác đặt hàng, thiết kế mở rộng cần bổ sung giai đoạn cấp ô, mở cho Shipper, xác nhận đặt hàng và đóng, rồi mới gửi thông báo sẵn sàng nhận.

2.4.2. Luồng nhận hàng

Cư dân vào ResidentScreen từ ứng dụng hoặc từ notification. Màn hình đăng ký FCM token, tải danh sách kiện bằng GET /packages/my và chỉ nhận dữ liệu của tài khoản hiện tại. Package được chọn từ thông báo được làm nổi bật để người dùng xác định đúng ô. OTP được cung cấp qua API cho chủ tài khoản và chỉ còn giá trị trong khoảng thời gian Backend quy định, khi hết hạn, người dùng yêu cầu refresh để thay bộ OTP.



Hình 2.4. Sơ đồ trình tự nghiệp vụ nhận hàng ở bản hiện tại

Khi yêu cầu mở tủ, Backend kiểm tra lần lượt role RESIDENT, cấu trúc OTP, sự tồn tại của package thuộc userId, trạng thái READY, thời hạn và giá trị hash. OTP đầu vào hiện được loại ký tự không phải số và lấy tối đa sáu chữ số trước khi kiểm tra. Vì vậy giới hạn sáu số được áp dụng sau chuẩn hóa, chưa phải kiểm tra nghiêm ngặt độ dài của chuỗi gốc. Nếu bất kỳ điều kiện nào không đạt, Backend trả lỗi và không tạo yêu cầu mở mới.

Trong SOFTWARE mode, redeem cập nhật package sang PICKED_UP ngay, ghi thời điểm nhận, xóa toàn bộ trường OTP và ghi NHẬN HÀNG. Nhánh này không cần ESP32. Trong ESP32 mode, Backend kiểm tra cấu hình device key, tạo hoặc dùng lại command OPEN đang phù hợp và trả WAITING_HARDWARE, commandId, commandStatus cùng locker. Ứng dụng đọc trạng thái command, command vẫn PENDING/CLAIMED thì tiếp tục chờ, DONE thì báo thành công, FAILED thì hiển thị lỗi.

ESP32 gọi GET /hardware/commands/next. Backend claim lệnh, firmware bật relay ứng với ô, giữ ON và gửi POST /hardware/commands/:commandId/complete. ACK thành công cho OPEN chỉ đánh dấu command DONE, package vẫn READY và OTP chưa bị xóa tại thời điểm này. ResidentScreen hiện dừng sau khi chờ OPEN thành công và tải lại dữ liệu, chưa gửi CLOSE.

Luồng hoàn tất được Backend thiết kế là: Cư dân lấy hàng, xác nhận kết thúc, client gửi CLOSE, ESP32 tắt relay và ACK, Backend chuyển package sang PICKED_UP, xóa OTP và ghi NHẬN HÀNG. API CLOSE và firmware đã hỗ trợ chuỗi này, nhưng cần bổ sung thao tác phía Cư dân. Cho đến khi nhận CLOSE ACK, ô vẫn bị xem là đã chiếm vì package còn READY, không được giải phóng chỉ dựa trên thông báo đã mở.

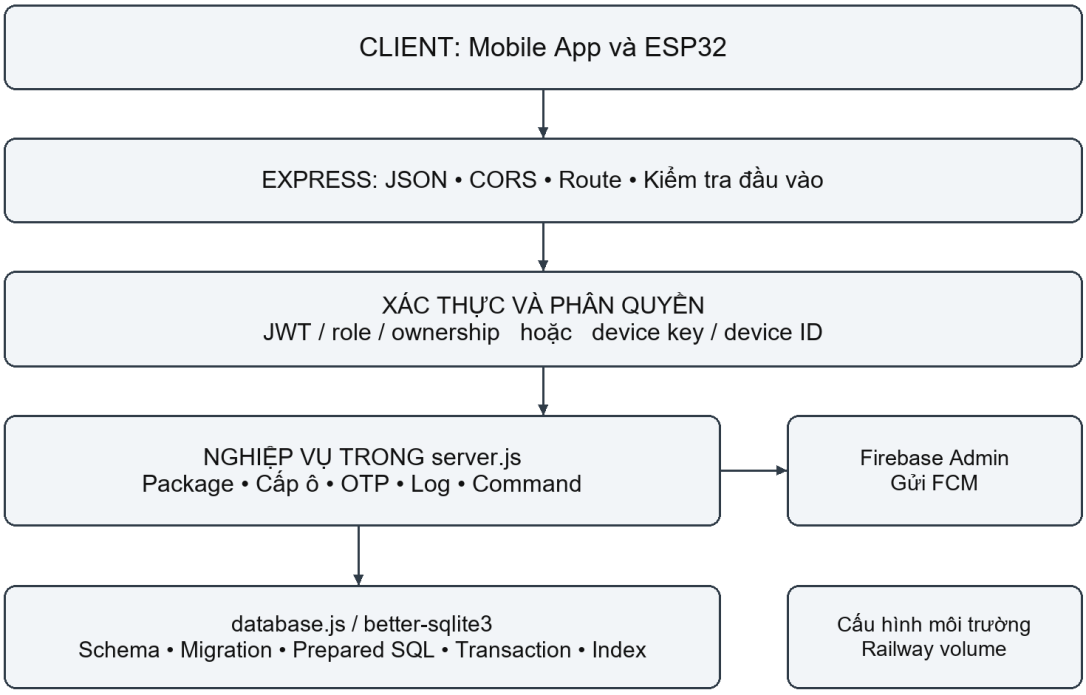
Thời gian chờ command ở ResidentScreen mặc định tối đa 45 giây. Hết thời gian chờ ở ứng dụng không đồng nghĩa lệnh bị hủy trong Backend: command có thể còn trong hàng đợi hoặc được thực hiện muộn khi thiết bị có mạng lại. Người dùng cần tải lại trạng thái, tránh hiểu timeout như bằng chứng chắc chắn relay chưa bật. Với prototype chưa có cảm biến, CLOSE ACK cũng chỉ xác nhận firmware đã thực hiện lệnh tắt relay, không chứng minh trực tiếp rằng cửa đóng kín hoặc hàng đã được lấy.

2.5. Thiết kế Backend và cơ sở dữ liệu

2.5.1. Kiến trúc Backend

Backend sử dụng Express với CORS và middleware đọc JSON. Các route được nhóm theo xác thực, thiết bị FCM, package, phần cứng và quản trị. authMiddleware kiểm tra Bearer JWT và gắn thông tin user vào request. Các handler tiếp tục kiểm tra role và quyền sở hữu. hardwareAuthMiddleware kiểm tra ESP32 mode cùng device key, rồi lấy device ID từ header hoặc giá trị mặc định để theo dõi thiết bị claim lệnh.

Nghiệp vụ trung tâm gồm chuẩn hóa dữ liệu, cấp tủ, tạo định danh, tạo và xác minh OTP, ghi nhật ký, gửi FCM, xếp command và xử lý ACK. Các helper như chooseAvailableLocker, createOtpBundle, queueOpenCommand, queueCloseCommand và finalizeHardwareCommand giúp tránh lặp quy tắc giữa Cư dân và Admin. Tuy nhiên đây là các nhóm trách nhiệm logic trong server.js, chưa phải kiến trúc nhiều tầng đã tách thành các module router/controller/service độc lập.



Hình 2.5. Sơ đồ kiến trúc Backend theo trách nhiệm xử lý

database.js mở tệp SQLite, bật journal_mode WAL và foreign_keys ON, tạo năm bảng cùng các index, đồng thời thực hiện migration cho schema cũ. Đường dẫn ưu tiên DB_PATH, nếu không có thì dùng RAILWAY_VOLUME_MOUNT_PATH hoặc DB_DIRECTORY, cuối cùng là tệp smartlocker.db cạnh mã Backend. Volume lưu bền vững cần được cấu hình trong triển khai thực tế, không chỉ dựa vào việc có biến trong mã nguồn.

Các truy vấn dùng prepared statement với tham số để tránh nối trực tiếp dữ liệu người dùng vào SQL. Claim command dùng transaction: lấy PENDING cũ nhất, cập nhật có điều kiện status vẫn PENDING, gắn thiết bị và tăng attempts. Xử lý ACK cũng dùng transaction để đồng bộ command, package và nhật ký ở nhánh CLOSE. Điều này

hạn chế trường hợp command hoàn tất nhưng trạng thái kiện chưa được cập nhật khi một thao tác bên trong thất bại.

Route tạo package hiện thực hiện chọn ô, INSERT package và ghi log bằng các câu lệnh riêng, nó chưa bọc toàn bộ thao tác trong một transaction cấp tủ và chưa có unique index chỉ cho một package READY trên mỗi locker. Với một tiến trình và các thao tác SQLite đồng bộ trước lần await gửi FCM, cửa sổ xen kẽ trong route được hạn chế, nhưng không thể suy rộng thành bảo đảm cấp tủ an toàn khi nhiều tiến trình cùng ghi. Khi mở rộng, cần giao dịch cấp tủ nguyên tử và ràng buộc duy nhất có điều kiện trên locker.

2.5.2. Mô hình dữ liệu

Mô hình dữ liệu tách tài khoản, thiết bị thông báo, kiện hàng, nhật ký và lệnh chấp hành thành năm bảng. Không có bảng lockers trong schema hiện tại, danh mục bốn ô nằm trong ALL_LOCKERS. Trạng thái trống hoặc có hàng được suy ra từ packages.status, còn thông tin thực thi relay được suy ra từ hardware_commands và ACK.

Bảng 2.3. Vai trò và ràng buộc của năm bảng SQLite

Bảng	Dữ liệu quản lý	Ràng buộc chính
users	Tài khoản và vai trò	PK id, UNIQUE(phone, role), CHECK role.
devices	Thiết bị nhận FCM	FK user_id → users.id, UNIQUE(user_id, fcm_token).
packages	Kiện, ô, người nhận và OTP	PK id, UNIQUE public_id/pickup_token, FK receiver_user_id, CHECK status.
activity_logs	Lịch sử giao nhận và can thiệp	UNIQUE event_key, CHECK action, liên kết public_id ở nghiệp vụ.
hardware_commands	Hàng đợi OPEN/CLOSE và ACK	CHECK action/status/requested_by, unique command đang hoạt động/package.

a. Bảng users

id là INTEGER PRIMARY KEY AUTOINCREMENT. phone, password_hash và role là TEXT NOT NULL, created_at là TEXT NOT NULL với mặc định CURRENT_TIMESTAMP. role được CHECK trong ADMIN, RESIDENT, SHIPPER và cặp (phone, role) có UNIQUE. Khóa nội bộ id dùng trong quan hệ dữ liệu, phone phục vụ định danh đăng nhập và tra người nhận, còn password_hash chỉ dùng để kiểm tra mật khẩu.

b. Bảng devices

id là khóa chính tự tăng, user_id là INTEGER NOT NULL tham chiếu users(id) với ON DELETE CASCADE. fcm_token là TEXT NOT NULL, platform mặc định android và updated_at mặc định CURRENT_TIMESTAMP. UNIQUE(user_id, fcm_token) ngăn đăng ký trùng cùng token cho cùng tài khoản. Quan hệ users–devices là một–nhiều để một Cư dân có thể nhận thông báo trên nhiều thiết bị. fcm_token không phải device key điều khiển ESP32 và không được dùng để cấp quyền mở tủ.

c. Bảng packages

id là khóa chính tự tăng, public_id là TEXT NOT NULL UNIQUE và được dùng trong URL, notification, log và command. receiver_user_id là INTEGER NOT NULL tham chiếu users(id), receiver_phone là TEXT NOT NULL lưu số người nhận, locker là TEXT NOT NULL. size mặc định M và sender mặc định Shipper trong schema, nhưng route tạo đơn lấy sender từ tài khoản đã xác thực. created_at mặc định CURRENT_TIMESTAMP, picked_up_at có thể NULL khi chưa nhận.

Các trường pickup_otp_hash, pickup_otp_enc, pickup_otp_iv, pickup_otp_tag và otp_expires_at đều là TEXT có thể NULL. Hash dùng xác minh, bộ enc/iv/tag dùng khôi phục OTP để hiển thị cho chủ tài khoản, còn thời hạn xác định khả năng sử dụng. pickup_token vẫn là TEXT NOT NULL UNIQUE để tương thích dữ liệu cũ, pickup_otp plaintext được giữ trong schema cho migration nhưng đơn mới ghi NULL. Khi package hoàn tất, các trường OTP và thời hạn bị xóa bằng NULL.

status là TEXT NOT NULL, mặc định READY và có CHECK trong READY, PICKED_UP, CANCELLED. READY chiếm ô theo thuật toán cấp tủ, PICKED_UP ghi nhận đã hoàn tất và không còn chiếm ô, CANCELLED mới là trạng thái được schema cho phép, chưa có luồng hủy qua API. Quan hệ users–packages là một–nhiều

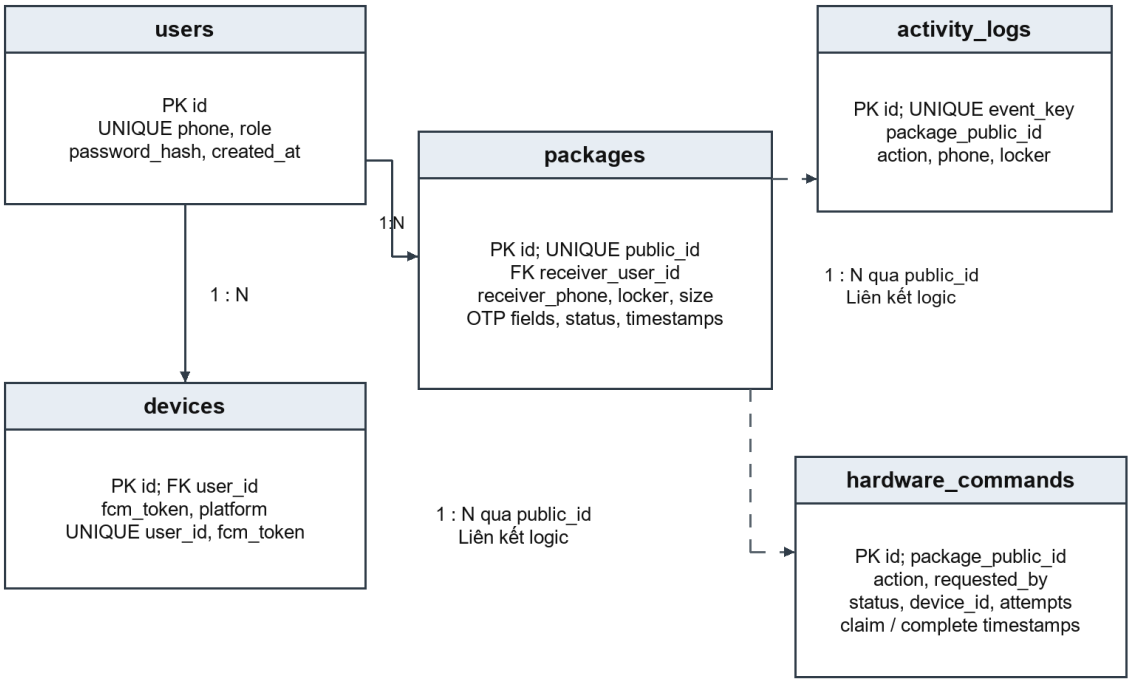
qua receiver_user_id, nhưng quy tắc người nhận phải mang role RESIDENT được kiểm tra ở nghiệp vụ thay vì khóa ngoại.

d. Bảng activity_logs

id là khóa chính tự tăng, event_key là TEXT NOT NULL UNIQUE. action bị CHECK trong GỬI HÀNG, NHẬN HÀNG và MỞ KHẨN CẤP. package_public_id có thể NULL, phone và locker là TEXT NOT NULL, created_at mặc định CURRENT_TIMESTAMP. Các event_key như PACKAGE_CREATED, PACKAGE_PICKED_UP hoặc FORCE_OPEN gắn với public_id giúp INSERT OR IGNORE không ghi lặp cùng loại sự kiện cho cùng kiện. Bảng giữ thông tin phone và locker tại sự kiện để đọc lịch sử thuận tiện.

e. Bảng hardware_commands

id là khóa chính tự tăng, package_public_id và locker là TEXT NOT NULL. action mặc định OPEN và có CHECK trong OPEN, CLOSE. requested_by có CHECK trong RESIDENT, ADMIN, requested_by_user_id là INTEGER có thể NULL. status mặc định PENDING và có CHECK trong PENDING, CLAIMED, DONE, FAILED. device_id, claimed_at, completed_at và error_message có thể NULL, attempts là INTEGER NOT NULL mặc định 0, created_at mặc định CURRENT_TIMESTAMP.



Nét liền: khóa ngoại trong schema. Nét đứt: liên kết do nghiệp vụ quản lý.

Hình 2.6. Sơ đồ quan hệ dữ liệu logic và khóa ngoại

Quan hệ users–devices và users–packages được bảo vệ bằng khóa ngoại. Liên kết từ packages.public_id đến activity_logs.package_public_id và hardware_commands.package_public_id là quan hệ logic một–nhiều, không có FOREIGN KEY trong schema. requested_by_user_id cũng chưa có khóa ngoại tới users. Khi bổ sung xóa tài khoản hoặc kiện, cần quy định giữ lịch sử, xử lý command đang chạy và kiểm soát bản ghi mồ côi, không thể mặc nhiên cho rằng SQLite đã ràng buộc các quan hệ logic này.

f. Chỉ mục và vòng đời dữ liệu

packages có index theo (receiver_user_id, status), receiver_phone, (locker, status) và pickup_token để phục vụ xem đơn, tra người nhận và cấp tủ. activity_logs có index created_at DESC cho lịch sử mới nhất. hardware_commands có index (status, id) để lấy lệnh theo thứ tự và (package_public_id, id DESC) để tìm lệnh gần nhất. Partial unique index trên package_public_id khi status thuộc PENDING hoặc CLAIMED bảo đảm mỗi package chỉ có một command đang hoạt động, index này không bảo đảm duy nhất cho ô tủ trong packages.

Vòng đời command là PENDING → CLAIMED → DONE hoặc FAILED. Khi CLAIMED quá hạn, command quay về PENDING để claim lại, đồng thời xóa device_id và claimed_at. attempts tăng khi claim thành công, không phải khi người dùng chỉ xem trạng thái. OPEN DONE giữ package READY, CLOSE DONE mới dẫn tới PICKED_UP. Trạng thái WAITING_HARDWARE, WAITING_CLOSE và ALREADY_OFF là nhãn phản hồi API, không nằm trong CHECK của packages.status.

CHƯƠNG 3. XÂY DỰNG PHẦN MỀM SMARTLOCKER

3.1. Công nghệ và môi trường phát triển

Hệ thống SmartLocker được xây dựng theo mô hình Client–Server, trong đó ứng dụng di động đóng vai trò giao diện tương tác với người dùng, Backend thực hiện xử lý nghiệp vụ và cơ sở dữ liệu đảm nhiệm việc lưu trữ thông tin. Các thành phần được kết nối thông qua hệ thống API nhằm phục vụ các chức năng đăng ký, đăng nhập, quản lý đơn hàng, xác thực nhận hàng, gửi thông báo và điều khiển thiết bị phần cứng.

Quá trình phát triển phần mềm sử dụng React Native kết hợp TypeScript để xây dựng ứng dụng Android, Node.js và Express.js để phát triển Backend, SQLite để quản lý dữ liệu và Firebase Cloud Messaging (FCM) để hỗ trợ gửi thông báo đến người dùng. Backend được triển khai trên nền tảng Railway nhằm cung cấp các dịch vụ API có thể truy cập thông qua Internet.

Môi trường phát triển chính được sử dụng là Android Studio. Công cụ này được dùng trong quá trình phát triển, quản lý mã nguồn, kết nối thiết bị Android và kiểm tra ứng dụng. Việc biên dịch và chạy ứng dụng được thực hiện thông qua React Native CLI. Hệ thống được kiểm thử trên cả máy ảo Android và thiết bị Android thực tế nhằm kiểm tra khả năng hoạt động của ứng dụng trong các môi trường khác nhau.

3.1.1. Tổng hợp công nghệ sử dụng

Bảng 3.1. Các công nghệ được sử dụng

Thành phần	Công nghệ hoặc công cụ	Vai trò
Ngôn ngữ ứng dụng	TypeScript	Xây dựng giao diện và xử lý logic ứng dụng
Framework Mobile	React Native 0.87.1	Phát triển ứng dụng Android
Thư viện giao diện	React 19.2.3	Xây dựng các thành phần giao diện
Điều hướng	React Navigation	Quản lý điều hướng giữa các màn hình
Lưu trữ cục bộ	AsyncStorage	Ghi nhớ tài khoản và dữ liệu cục bộ
Backend	Node.js	Môi trường thực thi JavaScript phía server
Framework Backend	Express.js 5.2.1	Xây dựng và xử lý REST API

Cơ sở dữ liệu	SQLite	Lưu trữ dữ liệu hệ thống
Truy cập cơ sở dữ liệu	better-sqlite3 13.0.3	Thực hiện truy vấn SQLite từ Backend
Thông báo	Firebase Cloud Messaging	Gửi và nhận thông báo
Firebase phía server	firebase-admin 14.4.0	Gửi thông báo từ Backend
Môi trường phát triển	Android Studio	Phát triển và kiểm thử ứng dụng Android
Công cụ chạy ứng dụng	React Native CLI	Biên dịch và chạy ứng dụng
Công cụ build	Gradle	Biên dịch ứng dụng Android
Công cụ đóng gói JS	Metro	Xử lý mã JavaScript trong môi trường React Native
Quản lý mã nguồn	Git, GitHub	Quản lý và cập nhật mã nguồn
Triển khai Backend	Railway	Vận hành Backend trên Internet
Giao tiếp	HTTP/HTTPS, REST API, JSON	Trao đổi dữ liệu giữa các thành phần

Từ những công nghệ và công cụ trên, hệ thống SmartLocker được tổ chức thành các thành phần có trách nhiệm riêng biệt, bao gồm giao diện người dùng, xử lý nghiệp vụ, lưu trữ dữ liệu, dịch vụ thông báo và giao tiếp với phần cứng.

Việc xây dựng ứng dụng bằng React Native kết hợp với Backend Node.js/Express giúp tách biệt quá trình xử lý phía thiết bị di động và phía máy chủ. Đồng thời, SQLite và Railway hỗ trợ quản lý dữ liệu và vận hành các API trên môi trường Internet, tạo nền tảng cho các chức năng được triển khai trong những mục tiếp theo.

3.2. Kiến trúc Mobile App

Ứng dụng di động SmartLocker được xây dựng bằng React Native kết hợp TypeScript, áp dụng cách tổ chức mã nguồn theo từng nhóm chức năng nhằm phân tách giao diện, điều hướng, quản lý trạng thái và giao tiếp với Backend. Kiến trúc này giúp các thành phần của ứng dụng có trách nhiệm tương đối độc lập, đồng thời vẫn có thể chia sẻ dữ liệu và phối hợp xử lý các nghiệp vụ gửi hàng, nhận hàng và quản trị hệ thống.

Ứng dụng đóng vai trò Client trong mô hình Client–Server. Các thao tác của người dùng được tiếp nhận và xử lý ban đầu trên ứng dụng, sau đó những yêu cầu liên quan đến tài khoản, đơn hàng, xác thực OTP hoặc điều khiển phần cứng được gửi đến Backend thông qua REST API. Backend thực hiện kiểm tra quyền truy cập, xử lý nghiệp vụ và trả kết quả để ứng dụng cập nhật giao diện tương ứng.

3.2.1. Cấu trúc tổ chức mã nguồn ứng dụng

Mã nguồn ứng dụng được tổ chức thành các nhóm tệp theo chức năng. Trong đó, các thành phần chính bao gồm tệp khởi tạo ứng dụng, điều hướng, quản lý trạng thái, các màn hình giao diện và những module hỗ trợ kết nối dịch vụ.

```
SmartLockerApp/
├── App.tsx
├── index.js
├── app.json
├── package.json
├── src/
│   ├── navigation/
│   │   └── AppNavigator.tsx
│   ├── context/
│   │   ├── AuthContext.tsx
│   │   └── LockerContext.tsx
│   ├── screens/
│   │   ├── HomeScreen.tsx
│   │   ├── LoginScreen.tsx
│   │   ├── RegisterScreen.tsx
│   │   ├── ShipperScreen.tsx
│   │   ├── ResidentScreen.tsx
│   │   ├── AdminScreen.tsx
│   │   └── HistoryScreen.tsx
│   ├── services/
│   │   ├── authApi.ts
│   │   └── firebaseMessaging.ts
```

```
│   ├── config/
│   │   └── api.ts
│   └── ai/
│       └── smartlockerOcr.ts
└── android/
    └── ...
```

Bảng 3.2. Vai trò của các nhóm tệp trong ứng dụng SmartLocker

Nhóm tệp	Chức năng
App.tsx	Khởi tạo cấu trúc ứng dụng và tích hợp các Provider
navigation/	Quản lý điều hướng giữa các màn hình
context/	Quản lý trạng thái xác thực và dữ liệu chia sẻ
screens/	Xây dựng giao diện theo từng chức năng và vai trò
services/	Tổ chức các hàm giao tiếp với API và dịch vụ hỗ trợ
config/	Quản lý cấu hình địa chỉ Backend
ai/	Chứa logic tích hợp xử lý nhận diện thông tin vận đơn
android/	Chứa cấu hình và thành phần native của ứng dụng Android

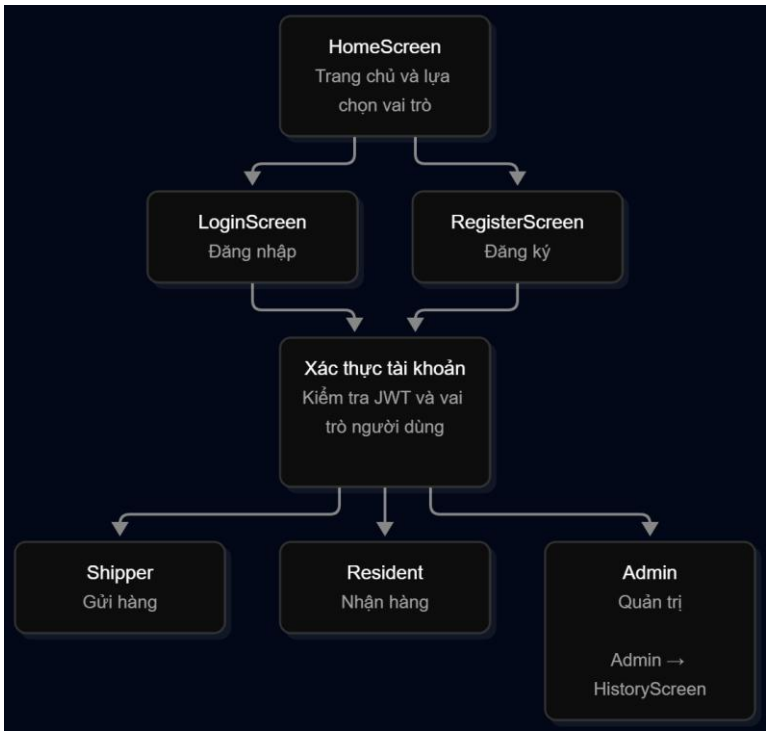
Việc phân chia mã nguồn theo các nhóm chức năng giúp hạn chế việc tập trung toàn bộ xử lý trong một tệp duy nhất. Khi cần cập nhật giao diện hoặc nghiệp vụ của một vai trò, quá trình chỉnh sửa có thể được thực hiện chủ yếu tại màn hình và các module liên quan mà không phải thay đổi toàn bộ ứng dụng.

3.2.2. Kiến trúc điều hướng giữa các màn hình

SmartLocker sử dụng React Navigation với cơ chế Native Stack Navigation để quản lý việc chuyển đổi giữa các màn hình.

Tệp AppNavigator.tsx khai báo những màn hình chính của ứng dụng, đồng thời tổ chức các thao tác điều hướng theo kết quả đăng nhập và vai trò tài khoản.

Ứng dụng hỗ trợ ba nhóm người dùng gồm Shipper, Cư dân và Admin. Sau khi xác thực thành công, người dùng được điều hướng đến màn hình chức năng tương ứng với vai trò của mình.



Hình 3.1. Sơ đồ điều hướng các màn hình theo vai trò người dùng.

Cơ chế điều hướng được xây dựng dựa trên thông tin vai trò đã xác thực, giúp người dùng truy cập đúng giao diện chức năng.

Ngoài điều hướng thông thường, AppNavigator.tsx còn tích hợp cơ chế xử lý thông báo từ Firebase Cloud Messaging.

Khi người dùng nhấn vào thông báo có dữ liệu đơn hàng, ứng dụng kiểm tra thông tin tài khoản hiện tại. Nếu người dùng đang đăng nhập đúng tài khoản Cư dân, ứng dụng thực hiện điều hướng tới ResidentScreen và truyền mã đơn hàng thông qua tham số packageId.

Trong trường hợp tài khoản đang đăng nhập không phù hợp hoặc chưa có phiên đăng nhập tương ứng, ứng dụng có cơ chế lưu tạm thông tin thông báo để tiếp tục xử lý sau khi người dùng đăng nhập đúng tài khoản.

3.2.3. Tổ chức giao tiếp giữa Mobile App và Backend

Giao tiếp giữa ứng dụng SmartLocker và Backend được thực hiện thông qua các REST API.

Địa chỉ máy chủ được khai báo trong tệp src/config/api.ts dưới dạng biến API_BASE_URL. Các màn hình và module dịch vụ sử dụng địa chỉ này để gửi yêu cầu tới Backend đang triển khai trên Railway.

Việc tập trung cấu hình địa chỉ Backend tại một vị trí giúp việc thay đổi môi trường kết nối thuận tiện hơn, hạn chế phải chỉnh sửa trực tiếp đường dẫn trong nhiều màn hình.

Ứng dụng sử dụng hàm fetch để thực hiện yêu cầu HTTP/HTTPS. Dữ liệu gửi và nhận chủ yếu sử dụng định dạng JSON.

Trong quá trình giao tiếp, ứng dụng xử lý phản hồi từ Backend, kiểm tra trạng thái yêu cầu và cập nhật giao diện hoặc hiển thị thông báo lỗi tùy theo kết quả nhận được.

Các chức năng xác thực như đăng ký, đăng nhập và xác minh phiên được hỗ trợ thông qua module authApi.ts. Những màn hình chức năng như Shipper, Resident và Admin cũng thực hiện các yêu cầu API phục vụ chức năng tương ứng.

Bảng 3.3. Một số nhóm giao tiếp giữa ứng dụng di động và Backend

Nhóm chức năng	API tiêu biểu	Mục đích
Xác thực	/auth/login, /auth/register	Đăng nhập, đăng ký tài khoản
Phiên người dùng	/auth/me	Kiểm tra JWT và thông tin tài khoản
Giao hàng	/packages	Tạo đơn hàng
Nhận hàng	/packages/my	Truy xuất đơn hàng của Cư dân
OTP	/packages/:id/otp/refresh	Cấp lại OTP
Mở tủ	/packages/:id/redeem	Xác minh OTP và gửi yêu cầu mở tủ
Tắt relay	/hardware/packages/:packageId/close	Yêu cầu ngắt relay sau khi mở tủ
Trạng thái lệnh	/hardware/commands/:id/status	Kiểm tra kết quả xử lý lệnh phần cứng
Quản trị	/admin/overview	Truy xuất trạng thái các ô tủ
Lịch sử	/admin/history	Truy xuất lịch sử giao dịch
Thiết bị FCM	/devices/register	Đăng ký thiết bị nhận thông báo

Các API trên được sử dụng tùy theo chức năng và quyền truy cập của từng vai trò. Việc kiểm tra JWT và quyền người dùng được thực hiện tại Backend, không chỉ phụ thuộc vào điều kiện hiển thị màn hình trên ứng dụng.

3.3. Trang chủ, đăng ký và đăng nhập

Trong hệ thống SmartLocker, các chức năng trang chủ, đăng ký và đăng nhập được xây dựng nhằm quản lý quá trình truy cập ứng dụng và xác định quyền sử dụng của từng nhóm người dùng. Hệ thống hỗ trợ ba vai trò chính gồm Shipper, Cư dân (Resident) và Quản trị viên (Admin). Mỗi vai trò được cung cấp giao diện và các chức năng nghiệp vụ tương ứng sau khi hoàn tất quá trình xác thực.

Các chức năng được phát triển trên nền tảng React Native kết hợp TypeScript, đồng thời sử dụng Backend Node.js/Express để xử lý thông tin tài khoản, xác minh mật khẩu, phân quyền và cấp mã xác thực phiên đăng nhập. Việc quản lý tài khoản được thực hiện tập trung tại Backend, giúp các thiết bị sử dụng chung dữ liệu người dùng thay vì phụ thuộc vào dữ liệu lưu trữ cục bộ.

Ngoài các chức năng xác thực cơ bản, ứng dụng còn tích hợp cơ chế ghi nhớ nhiều tài khoản trên cùng một thiết bị nhằm hỗ trợ chuyển đổi tài khoản thuận tiện trong quá trình sử dụng và kiểm thử.

3.3.1. Xây dựng chức năng đăng ký tài khoản

Chức năng đăng ký được triển khai trong tệp RegisterScreen.tsx. Hệ thống sử dụng một màn hình đăng ký chung cho cả ba vai trò, trong đó nội dung biểu mẫu và điều kiện xác thực được thay đổi dựa trên nhóm tài khoản mà người dùng lựa chọn.

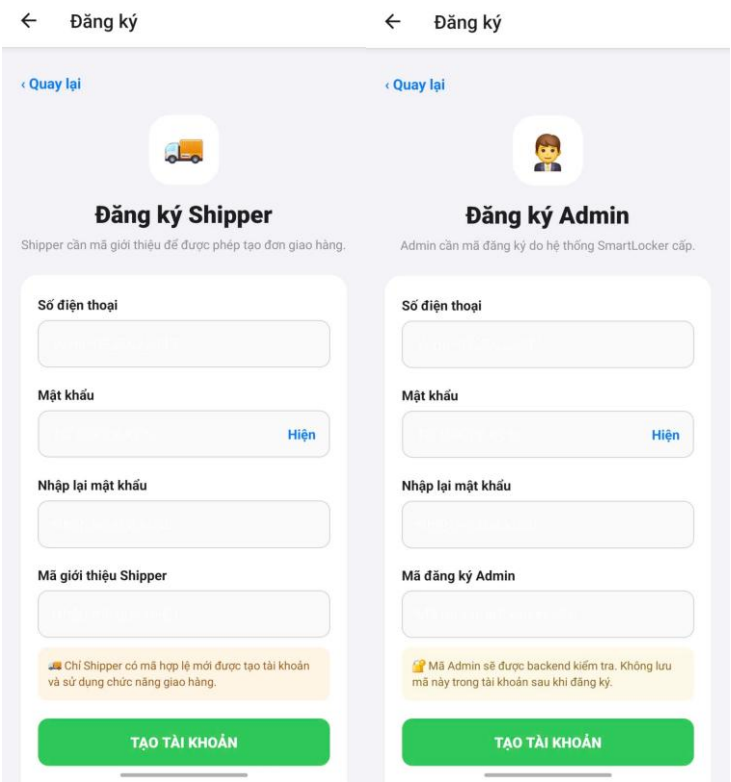
Các thông tin cơ bản cần cung cấp bao gồm số điện thoại, mật khẩu và xác nhận mật khẩu. Đối với vai trò Admin và Shipper, hệ thống yêu cầu thêm mã đăng ký riêng để hạn chế việc tạo tài khoản không được phép.

Bảng 3.4. Thông tin đăng ký theo từng vai trò

Vai trò	Thông tin đăng ký	Điều kiện bổ sung
Resident	Số điện thoại, mật khẩu, nhập lại mật khẩu	Không yêu cầu mã vai trò
Shipper	Số điện thoại, mật khẩu, nhập lại mật khẩu	Yêu cầu mã giới thiệu Shipper
Admin	Số điện thoại, mật khẩu, nhập lại mật khẩu	Yêu cầu mã đăng ký Admin

Trước khi gửi yêu cầu đăng ký đến Backend, ứng dụng thực hiện kiểm tra các dữ liệu đầu vào nhằm hạn chế những yêu cầu không hợp lệ. Số điện thoại được chuẩn hóa và kiểm tra theo định dạng gồm 10 chữ số, bắt đầu bằng số 0. Mật khẩu phải có độ dài tối thiểu 6 ký tự và trùng khớp với nội dung xác nhận mật khẩu.

Đối với tài khoản Admin và Shipper, ứng dụng kiểm tra việc nhập mã đăng ký tương ứng trước khi gửi yêu cầu. Giá trị mã được Backend xác minh dựa trên cấu hình hệ thống, thay vì chỉ kiểm tra tại giao diện.



Hình 3.2. Giao diện đăng ký tài khoản Shipper và Admin.

Sau khi các dữ liệu đầu vào được kiểm tra, ứng dụng gửi yêu cầu đăng ký đến Backend thông qua API POST /auth/register. Backend tiếp nhận thông tin, kiểm tra số điện thoại, vai trò, điều kiện đăng ký và sự tồn tại của tài khoản trong cơ sở dữ liệu.

Nếu thông tin hợp lệ, Backend tiến hành tạo tài khoản mới và lưu dữ liệu vào SQLite. Mật khẩu được xử lý bằng thư viện bcryptjs trước khi lưu trữ nhằm tránh lưu mật khẩu trực tiếp dưới dạng văn bản thông thường.

Sau khi đăng ký thành công, Backend trả về thông tin người dùng cùng JWT. Ứng dụng cập nhật trạng thái xác thực, ghi nhớ tài khoản trên thiết bị và điều hướng đến màn hình chức năng tương ứng với vai trò vừa đăng ký.

3.3.2. Xây dựng chức năng đăng nhập và xác thực tài khoản

Chức năng đăng nhập được triển khai trong tệp LoginScreen.tsx, cho phép người dùng truy cập hệ thống bằng số điện thoại và mật khẩu. Màn hình đăng nhập nhận thông tin vai trò từ trang chủ để xác định nhóm tài khoản cần xác thực.

Ứng dụng sử dụng biểu mẫu đăng nhập chung cho ba vai trò, giúp thống nhất cách thức tương tác và hạn chế việc xây dựng nhiều màn hình có chức năng tương tự nhau.

Khi người dùng nhập thông tin đăng nhập và xác nhận, ứng dụng gửi yêu cầu đến API POST /auth/login. Dữ liệu gửi bao gồm số điện thoại, mật khẩu và vai trò tài khoản.

Sau khi tiếp nhận yêu cầu, Backend tiến hành truy vấn cơ sở dữ liệu để tìm tài khoản có số điện thoại và vai trò tương ứng. Nếu tài khoản tồn tại, hệ thống sử dụng hàm bcrypt.compare() để đối chiếu mật khẩu người dùng cung cấp với giá trị băm đã được lưu trữ.

Trường hợp xác thực thành công, Backend tạo JWT chứa các thông tin nhận diện phiên đăng nhập như mã người dùng, số điện thoại và vai trò. JWT được gửi về ứng dụng để phục vụ các yêu cầu xác thực tiếp theo.

Ứng dụng sử dụng AuthContext để cập nhật trạng thái phiên hiện tại và điều hướng người dùng đến màn hình nghiệp vụ phù hợp.

Quy trình xác thực tài khoản được thực hiện theo các bước chính: tiếp nhận thông tin đăng nhập, gửi yêu cầu đến Backend, kiểm tra tài khoản trong SQLite, xác minh mật khẩu, cấp JWT và điều hướng theo vai trò. Nếu thông tin không hợp lệ, Backend trả về thông báo lỗi để ứng dụng hiển thị cho người dùng.

Bảng 3.5. Phân quyền điều hướng sau đăng nhập

Vai trò	Màn hình đích	Chức năng chính
Shipper	ShipperScreen	Gửi hàng và tạo đơn
Resident	ResidentScreen	Theo dõi đơn, xác thực OTP và nhận hàng
Admin	AdminScreen	Theo dõi trạng thái ô tủ và lịch sử giao dịch

Ngoài việc kiểm soát quyền truy cập tại giao diện, Backend còn sử dụng JWT và các điều kiện kiểm tra vai trò để bảo vệ những API quan trọng. Điều này giúp ngăn

người dùng truy cập trái quyền vào các chức năng nghiệp vụ, kể cả khi yêu cầu được gửi trực tiếp đến máy chủ mà không thông qua ứng dụng.

3.4. Xây dựng chức năng Shipper

Chức năng Shipper được xây dựng nhằm hỗ trợ nhân viên giao hàng thực hiện quá trình tạo đơn và phân bổ kiện hàng vào hệ thống tủ thông minh SmartLocker. Đây là một trong những chức năng nghiệp vụ chính của ứng dụng, đóng vai trò khởi tạo thông tin kiện hàng để các thành phần khác trong hệ thống tiếp tục xử lý.

Trong hệ thống, Shipper sử dụng tài khoản đã được xác thực để truy cập giao diện giao hàng, nhập hoặc nhận diện số điện thoại người nhận, kiểm tra thông tin và gửi yêu cầu tạo đơn đến Backend. Sau khi tiếp nhận yêu cầu, Backend thực hiện kiểm tra tài khoản Cư dân, lựa chọn ô tủ phù hợp, tạo thông tin kiện hàng, lưu dữ liệu và gửi thông báo đến người nhận thông qua Firebase Cloud Messaging.

Chức năng được phát triển trong tệp ShipperScreen.tsx, sử dụng React Native để xây dựng giao diện và kết nối với Backend thông qua REST API. Ngoài phương thức nhập số điện thoại thủ công, màn hình còn tích hợp chức năng camera kết hợp OCR nhằm hỗ trợ nhận diện thông tin người nhận từ hình ảnh vận đơn. Phần thuật toán nhận diện được trình bày riêng trong nội dung về trí tuệ nhân tạo của dự án.

3.4.1. Tích hợp camera và chức năng nhận diện số điện thoại người nhận

Nhằm giảm thao tác nhập liệu thủ công, màn hình Shipper được tích hợp chức năng camera để thu thập hình ảnh vận đơn và hỗ trợ nhận diện số điện thoại người nhận.

Ứng dụng sử dụng thư viện react-native-vision-camera để truy cập camera trên thiết bị Android. Trước khi sử dụng, hệ thống kiểm tra quyền truy cập camera và tình trạng thiết bị camera. Khi người dùng thực hiện thao tác chụp ảnh, hình ảnh được chuyển đến module xử lý OCR để trích xuất thông tin văn bản. Trong mã nguồn, ảnh đầu vào được xử lý kích thước thông qua thư viện react-native-image-resizer trước khi đưa vào quá trình nhận diện.

Kết quả OCR được sử dụng để xác định số điện thoại có khả năng thuộc về người nhận hàng. Sau đó, số điện thoại được điền vào trường thông tin người nhận trên giao diện Shipper.

Người dùng có thể kiểm tra và chỉnh sửa kết quả nhận diện trước khi xác nhận tạo đơn. Điều này giúp hạn chế ảnh hưởng của các trường hợp hình ảnh không rõ nét, số điện thoại bị nhận diện sai hoặc vận đơn chứa nhiều số điện thoại khác nhau.

Sau khi quá trình nhận diện hoàn tất, ứng dụng hiển thị số điện thoại đã được trích xuất để Shipper kiểm tra trước khi gửi yêu cầu tạo đơn.

3.4.2. Xây dựng chức năng tạo đơn hàng

Chức năng tạo đơn hàng được thực hiện khi Shipper xác nhận thông tin người nhận trên giao diện. Ứng dụng gửi yêu cầu đến Backend thông qua API:

POST /packages

Yêu cầu được gửi kèm JWT của Shipper để Backend xác minh quyền thực hiện. Dữ liệu nghiệp vụ chính bao gồm số điện thoại người nhận và kích thước kiện hàng. Backend tiếp nhận các thông tin và thực hiện các bước xử lý nghiệp vụ theo thứ tự.

Đầu tiên, Backend kiểm tra quyền truy cập của Shipper và xác minh tài khoản người nhận. Sau đó, hệ thống xác định các ô tủ đang trống, lựa chọn ô tủ phù hợp và tạo thông tin đơn hàng mới.

Khi tạo đơn, Backend sinh mã định danh công khai cho kiện hàng và tạo OTP nhận hàng gồm 6 chữ số. Thông tin đơn hàng được lưu vào cơ sở dữ liệu SQLite với trạng thái READY, biểu thị kiện hàng đang trong quá trình chờ người nhận thực hiện thao tác lấy hàng.

Backend đồng thời ghi nhận sự kiện gửi hàng vào lịch sử giao dịch để phục vụ chức năng quản trị và theo dõi hoạt động.

Sau khi dữ liệu được xử lý, Backend gửi kết quả về ứng dụng, bao gồm thông tin đơn hàng và mã ô tủ đã được phân bổ.

Nút xác nhận tạo đơn trên giao diện được quản lý bằng trạng thái đang xử lý nhằm hạn chế việc người dùng nhấn nhiều lần trong khi yêu cầu trước đó chưa hoàn tất.

3.5. Xây dựng chức năng Cư dân

Chức năng Cư dân (Resident) được xây dựng nhằm hỗ trợ người nhận quản lý các kiện hàng đã được gửi đến hệ thống SmartLocker và thực hiện quá trình nhận hàng thông qua xác thực OTP. Đây là thành phần trực tiếp kết nối giữa người nhận hàng, Backend và mô hình tủ thông minh, bảo đảm các thao tác nhận hàng được thực hiện theo đúng tài khoản và quyền truy cập.

Trong hệ thống, khi Shipper tạo đơn hàng thành công, Backend lưu thông tin kiện hàng vào cơ sở dữ liệu SQLite và gửi thông báo đến thiết bị của Cư dân thông qua Firebase Cloud Messaging (FCM). Sau khi đăng nhập, Cư dân có thể theo dõi danh sách đơn hàng đang chờ nhận, xem mã OTP được cung cấp tự động, kiểm tra thời gian hiệu lực và thực hiện yêu cầu mở tủ.

Đối với chế độ ESP32, yêu cầu mở tủ được xác thực tại Backend trước khi tạo lệnh OPEN cho relay tương ứng. Trong phiên bản hiện tại, ResidentScreen theo dõi kết quả OPEN và tải lại dữ liệu; màn hình Cư dân chưa gửi lệnh CLOSE sau khi lấy hàng. Backend và firmware đã hỗ trợ CLOSE, nhưng thao tác xác nhận hoàn tất phía Cư dân cần được bổ sung để kết thúc đầy đủ quá trình nhận hàng.

Chức năng được xây dựng chủ yếu trong tệp ResidentScreen.tsx, sử dụng React Native để tổ chức giao diện, React Hooks để quản lý trạng thái và REST API để trao đổi dữ liệu với Backend.

3.5.1. Xây dựng chức năng hiển thị OTP tự động

Một trong những chức năng quan trọng của màn hình Cư dân là hiển thị OTP nhận hàng tự động, thay thế cho phương thức yêu cầu người dùng nhập mã thủ công.

Khi đơn hàng được tạo, Backend sinh OTP gồm 6 chữ số và liên kết mã này với kiện hàng tương ứng. OTP được sử dụng để xác minh quyền nhận hàng trước khi hệ thống thực hiện yêu cầu mở tủ.

Khi Cư dân truy xuất danh sách đơn hàng, Backend kiểm tra quyền sở hữu, trạng thái kiện hàng và thời hạn của OTP. Nếu mã còn hiệu lực, Backend cung cấp thông tin OTP để ứng dụng hiển thị.

Tại giao diện, OTP được chia thành sáu ô số riêng biệt. Các ô này được xây dựng bằng thành phần View và Text của React Native, có chức năng hiển thị mã do Backend trả về chứ không yêu cầu người dùng tự nhập.

Bên dưới khu vực OTP, ứng dụng hiển thị thông báo xác nhận mã đã được tự động điền, đồng thời cung cấp nút mở tủ khi các điều kiện nhận hàng được đáp ứng.

Cơ chế này giúp đơn giản hóa thao tác nhận hàng vì người dùng không cần ghi nhớ hoặc nhập lại mã OTP. Tuy nhiên, quá trình xác thực vẫn được thực hiện tại Backend nhằm bảo đảm mã OTP hợp lệ và thuộc đúng kiện hàng của Cư dân.

3.5.2. Quản lý thời hạn và cấp lại OTP

OTP được thiết kế có thời gian hiệu lực nhằm hạn chế việc sử dụng mã nhận hàng trong thời gian dài. Theo cấu hình Backend, thời gian hiệu lực mặc định là 15 phút nếu không được thay đổi thông qua biến môi trường.

Ứng dụng sử dụng thông tin otpExpiresAt do Backend cung cấp để xác định thời điểm hết hạn. Đồng thời, màn hình sử dụng bộ đếm thời gian cập nhật mỗi giây nhằm tính toán và hiển thị thời gian còn lại.

Khi OTP còn hiệu lực, ứng dụng hiển thị mã nhận hàng và nút mở tủ. Khi thời gian hiệu lực kết thúc, giao diện chuyển sang trạng thái thông báo OTP đã hết hạn và cung cấp chức năng cấp lại mã. Để cấp OTP mới, ứng dụng gửi yêu cầu:

POST /packages/:id/otp/refresh

Authorization: Bearer <JWT_TOKEN>

Backend kiểm tra quyền sở hữu kiện hàng và trạng thái đơn trước khi thực hiện tạo OTP mới. Sau khi nhận kết quả thành công, ứng dụng tải lại danh sách kiện hàng để cập nhật mã OTP và thời gian hiệu lực tương ứng.

Việc tích hợp chức năng cấp lại OTP giúp người dùng tiếp tục thực hiện nhận hàng khi mã trước đó không còn hiệu lực mà không cần yêu cầu Shipper tạo lại đơn hàng.

3.5.3. Xây dựng chức năng mở tủ nhận hàng

Sau khi OTP được hiển thị và xác định còn hiệu lực, Cư dân có thể thực hiện thao tác mở tủ bằng cách nhấn nút MỞ TỦ trên thẻ kiện hàng. Khi nhận được thao tác, ứng dụng gửi yêu cầu đến Backend thông qua API:

POST /packages/:id/redeem

Authorization: Bearer <JWT_TOKEN>

Content-Type: application/json

Backend tiếp nhận yêu cầu và thực hiện những bước kiểm tra cần thiết, bao gồm xác thực JWT, kiểm tra quyền sở hữu kiện hàng, kiểm tra trạng thái đơn hàng và xác minh giá trị OTP.

Nếu các điều kiện hợp lệ và hệ thống đang sử dụng chế độ điều khiển phần cứng, Backend tạo lệnh OPEN tương ứng với ô tủ cần mở.

Thiết bị ESP32 kết nối với Backend thông qua Wi-Fi và định kỳ gửi yêu cầu lấy lệnh. Khi nhận được lệnh OPEN, ESP32 xác định kênh relay tương ứng và thực hiện kích hoạt relay để mở tủ.

Sau khi thực thi lệnh, ESP32 gửi phản hồi xác nhận đến Backend. Ứng dụng Cư dân kiểm tra trạng thái lệnh thông qua API:

GET /hardware/commands/:id/status

Authorization: Bearer <JWT_TOKEN>

Ứng dụng tiếp tục kiểm tra trạng thái cho đến khi nhận được kết quả xác nhận hoàn thành, nhận thông báo thất bại hoặc hết thời gian chờ.

Trong mã nguồn ResidentScreen.tsx, thời gian chờ xác nhận lệnh phần cứng được cấu hình mặc định là 45 giây, với chu kỳ kiểm tra trạng thái khoảng 1 giây. Khi Backend trả về trạng thái lệnh DONE, ứng dụng hiển thị thông báo mở tủ thành công cho người dùng.

3.6. Xây dựng chức năng Quản trị viên và lịch sử

Chức năng Quản trị viên (Admin) được xây dựng nhằm hỗ trợ việc giám sát và quản lý hoạt động của hệ thống SmartLocker. Đây là thành phần cung cấp thông tin tổng hợp về tình trạng sử dụng các ô tủ, các kiện hàng đang chờ nhận và lịch sử giao dịch đã phát sinh trong hệ thống.

Trong quá trình vận hành, các dữ liệu liên quan đến tài khoản, đơn hàng và trạng thái ô tủ được quản lý tập trung tại Backend và cơ sở dữ liệu SQLite. Ứng dụng Admin truy xuất những thông tin này thông qua REST API để hiển thị tình trạng hoạt động của hệ thống, giúp người quản trị theo dõi việc sử dụng các ô tủ và thực hiện thao tác xử lý khi cần thiết.

Ngoài chức năng giám sát, Admin còn được cung cấp quyền mở tủ khẩn cấp đối với kiện hàng đang chờ nhận. Trong phiên bản tích hợp phần cứng, yêu cầu mở tủ được điều phối thông qua Backend và thiết bị ESP32. Sau khi thiết bị xác nhận thực hiện lệnh, người quản trị có thể thực hiện thao tác tắt relay để hoàn tất quá trình xử lý.

Nhóm chức năng quản trị được xây dựng chủ yếu trong hai tệp AdminScreen.tsx và HistoryScreen.tsx, kết hợp React Native, React Hooks và các API quản trị tại Backend.

3.6.1. Xây dựng bảng thống kê tổng quan

Bảng thống kê tổng quan được xây dựng nhằm cung cấp cho Admin thông tin về tình trạng sử dụng các ô tủ và số lượng giao dịch được ghi nhận trong hệ thống.

Khi màn hình quản trị được truy cập, ứng dụng gửi yêu cầu đến Backend thông qua API:

GET /admin/overview

Authorization: Bearer <JWT_TOKEN>

Backend tiếp nhận yêu cầu, xác minh quyền Admin và truy xuất các dữ liệu liên quan từ cơ sở dữ liệu SQLite.

Dữ liệu sau khi xử lý được trả về ứng dụng dưới định dạng JSON, bao gồm thông tin tổng hợp và trạng thái từng ô tủ.

Các chỉ số được hiển thị trên bảng thống kê gồm tổng số ô tủ, số ô đang sử dụng, số ô còn trống và số giao dịch.

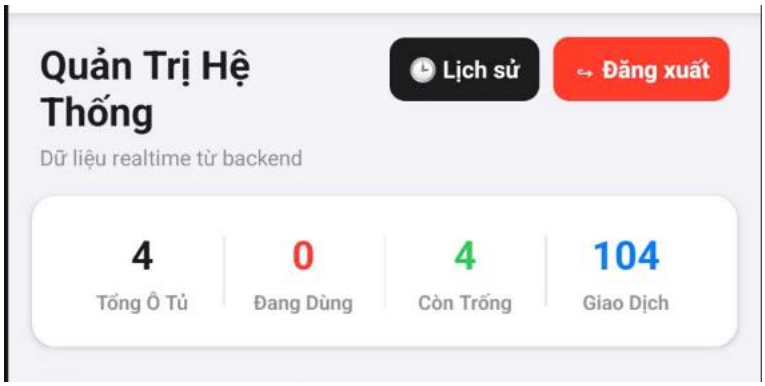
Bảng 3.6. Các chỉ số thống kê trên màn hình Admin

Chỉ số	Nội dung
Tổng ô tủ	Tổng số ô tủ được cấu hình trong hệ thống
Đang sử dụng	Số ô tủ có kiện hàng đang chờ nhận
Còn trống	Số ô tủ chưa được phân bổ cho đơn hàng đang chờ nhận
Giao dịch	Tổng số bản ghi hoạt động được thống kê từ lịch sử hệ thống

Trạng thái sử dụng của các ô tủ được xác định dựa trên những đơn hàng đang có trạng thái READY. Nếu một ô tủ đang được gán cho đơn hàng chưa hoàn tất nhận hàng, ô tủ đó được xác định là đang sử dụng.

Ngược lại, khi không tồn tại đơn hàng READY tương ứng, ô tủ được xác định là còn trống theo dữ liệu nghiệp vụ.

Đối với chỉ số giao dịch, hệ thống thống kê dựa trên những bản ghi trong bảng lịch sử hoạt động. Vì một đơn hàng có thể phát sinh nhiều sự kiện như gửi hàng và nhận hàng, số lượng giao dịch không nhất thiết bằng số lượng kiện hàng đã tạo.



Hình 3.3. Bảng thống kê tổng quan tình trạng hoạt động của SmartLocker.

Việc hiển thị các chỉ số dưới dạng bảng tổng quan giúp Admin nhanh chóng nắm được tình trạng sử dụng hệ thống mà không cần kiểm tra riêng từng đơn hàng.

3.6.2. Xây dựng chức năng giám sát trạng thái ô tủ

Bên cạnh bảng thống kê, hệ thống còn cung cấp danh sách các ô tủ để người quản trị theo dõi tình trạng sử dụng.

Với mô hình của nhóm hiện tại, SmartLocker sử dụng bốn ô tủ vật lý gồm A01, A02, B01 và B05. Mỗi ô tủ được gắn với một mã định danh để phục vụ quá trình phân bổ đơn hàng và điều khiển phần cứng. Thông tin trạng thái được Backend tổng hợp dựa trên dữ liệu đơn hàng trong SQLite.

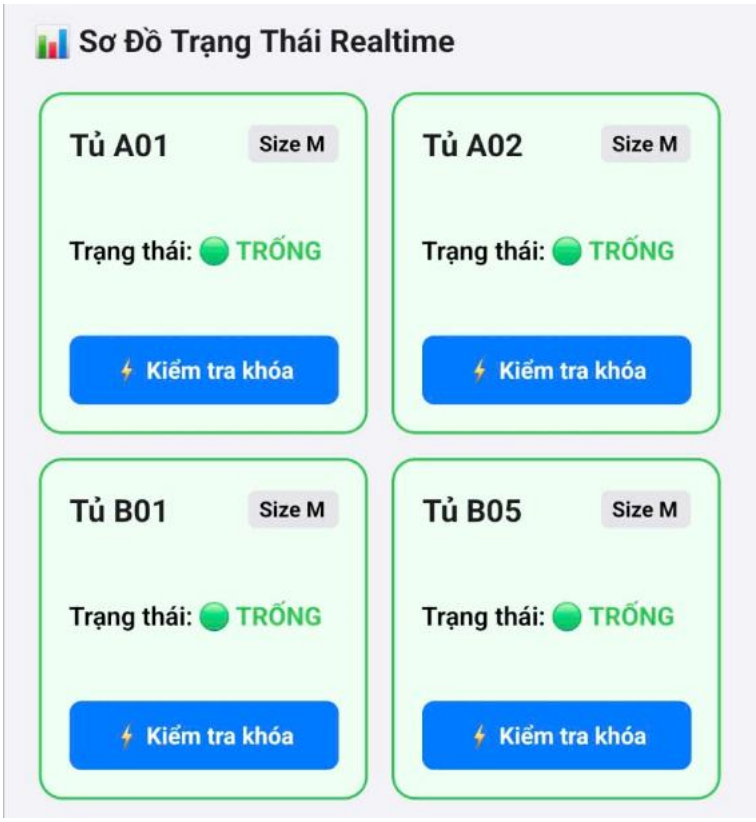
Các trạng thái nghiệp vụ chính được hiển thị bao gồm:

TRỐNG: Ô tủ không có đơn hàng đang chờ nhận.

CÓ HÀNG: Ô tủ đang được liên kết với một đơn hàng có trạng thái READY.

Đối với ô tủ đang sử dụng, giao diện có thể hiển thị thêm thông tin liên quan như mã đơn hàng, số điện thoại người nhận và thông tin người gửi.

Danh sách ô tủ được tổ chức dưới dạng các thẻ giao diện, giúp Admin dễ dàng phân biệt giữa những ô đang sử dụng và những ô còn trống.



Hình 3.4. Giao diện giám sát trạng thái các ô tủ SmartLocker.

Khi người dùng kéo để làm mới hoặc truy cập lại màn hình, ứng dụng thực hiện yêu cầu truy xuất dữ liệu từ Backend để cập nhật trạng thái ô tủ.

Cơ chế này giúp thông tin hiển thị phản ánh trạng thái nghiệp vụ đã được lưu trữ trên máy chủ. Tuy nhiên, trạng thái TRỐNG hoặc CÓ HÀNG được xác định dựa trên dữ liệu đơn hàng, không phải kết quả đo trực tiếp từ cảm biến phát hiện vật thể bên trong tủ. Vì vậy, trạng thái hiển thị trên giao diện Admin thể hiện tình trạng phân bổ ô tủ theo phần mềm.

3.6.3. Xây dựng chức năng mở tủ khẩn cấp

Chức năng mở tủ khẩn cấp được xây dựng nhằm hỗ trợ Admin xử lý những trường hợp cần can thiệp vào kiện hàng đang chờ nhận.

Trong quá trình vận hành, có thể phát sinh những tình huống người nhận không thể thực hiện thao tác nhận hàng theo quy trình thông thường. Khi đó, tài khoản Admin được cấp quyền thực hiện yêu cầu mở tủ đối với kiện hàng tương ứng.

Trên giao diện quản trị, khi một ô tủ đang ở trạng thái có hàng, Admin có thể lựa chọn chức năng mở khẩn cấp.

Trước khi thực hiện, ứng dụng hiển thị hộp thoại xác nhận nhằm hạn chế việc mở tủ ngoài ý muốn.

Sau khi xác nhận, ứng dụng gửi yêu cầu đến Backend thông qua API:

POST /admin/packages/:id/force-open

Authorization: Bearer <JWT_TOKEN>

Backend kiểm tra quyền Admin, xác định kiện hàng và trạng thái hiện tại trước khi xử lý yêu cầu.

Đối với phiên bản tích hợp phần cứng, Backend thực hiện điều phối lệnh OPEN đến ESP32 để kích hoạt relay của ô tủ tương ứng.

ESP32 truy xuất lệnh điều khiển từ Backend thông qua kết nối Wi-Fi. Khi nhận được lệnh mở, thiết bị xác định kênh relay và thực hiện kích hoạt.

Sau khi thực thi, ESP32 gửi phản hồi xác nhận đến Backend. Ứng dụng kiểm tra trạng thái lệnh để xác định kết quả xử lý.

Khi lệnh OPEN được xác nhận hoàn thành, ứng dụng hiển thị thông báo mở tủ thành công.

3.6.4. Xây dựng chức năng lịch sử giao dịch

Chức năng lịch sử giao dịch được xây dựng trong tệp HistoryScreen.tsx, giúp Admin theo dõi các thao tác nghiệp vụ đã phát sinh trong hệ thống SmartLocker.

Dữ liệu lịch sử được lưu trữ tập trung trong cơ sở dữ liệu SQLite tại Backend, thay vì chỉ lưu cục bộ trên thiết bị đang sử dụng.

Khi Admin truy cập màn hình lịch sử, ứng dụng gửi yêu cầu:

GET /admin/history

Authorization: Bearer <JWT_TOKEN>

Backend kiểm tra quyền Admin và truy xuất dữ liệu từ bảng activity_logs.

Các bản ghi được sắp xếp theo thời gian để những hoạt động mới nhất xuất hiện trước. API giới hạn tối đa 500 bản ghi lịch sử trong một lần trả về.

Mỗi bản ghi lịch sử chứa các thông tin phục vụ việc theo dõi và đối chiếu hoạt động, bao gồm loại thao tác, mã đơn hàng, mã ô tủ, số điện thoại liên quan và thời gian phát sinh.

Bảng 3.7. Các loại sự kiện nghiệp vụ được ghi nhận trong lịch sử SmartLocker

Loại sự kiện	Nội dung
GỬI HÀNG	Ghi nhận khi Backend tạo đơn hàng thành công
NHẬN HÀNG	Ghi nhận khi kiện hàng được hoàn tất nhận
MỞ KHẨN CẤP	Ghi nhận khi Admin thực hiện mở tủ khẩn cấp thành công

Các sự kiện được Backend ghi nhận trong quá trình xử lý nghiệp vụ, giúp lịch sử giao dịch phản ánh những thay đổi đã được thực hiện đối với đơn hàng.

Đối với mỗi sự kiện, giao diện lịch sử hiển thị các thông tin cần thiết để người quản trị xác định thời điểm, ô tủ và kiện hàng liên quan.

Bên cạnh việc hiển thị danh sách, màn hình lịch sử còn hỗ trợ làm mới dữ liệu khi được truy cập hoặc khi người dùng thực hiện thao tác kéo để cập nhật.

Nhờ được lưu trữ tại Backend, lịch sử giao dịch có thể được truy xuất từ các thiết bị đăng nhập bằng tài khoản Admin mà không phụ thuộc vào dữ liệu cục bộ của một điện thoại cụ thể.

3.7. Cơ chế OTP và phân bổ ô tủ

Cơ chế xác thực OTP và phân bổ ô tủ là hai thành phần nghiệp vụ quan trọng trong hệ thống SmartLocker, đảm nhiệm việc kiểm soát quyền nhận hàng và quản lý tài nguyên lưu trữ. Hai cơ chế này được xử lý tập trung tại Backend Node.js/Express nhằm bảo đảm thông tin kiện hàng, tài khoản người nhận và trạng thái sử dụng ô tủ được quản lý thống nhất.

Trong quá trình giao hàng, khi Shipper tạo đơn thành công, Backend kiểm tra tài khoản Cư dân, xác định ô tủ phù hợp và tạo mã OTP cho kiện hàng. Thông tin này được lưu trữ trong cơ sở dữ liệu SQLite để phục vụ quá trình theo dõi và nhận hàng.

Khi Cư dân thực hiện yêu cầu mở tủ, Backend kiểm tra quyền sở hữu kiện hàng, trạng thái đơn hàng, thời hạn và tính hợp lệ của OTP trước khi cho phép thực hiện thao tác điều khiển phần cứng.

Đối với mô hình SmartLocker hiện tại, hệ thống sử dụng bốn ô tủ vật lý được điều khiển thông qua ESP32 và module relay bốn kênh. Việc phân bổ ô tủ dựa trên trạng thái đơn hàng trong cơ sở dữ liệu, giúp hạn chế tình trạng một ô tủ được cấp đồng thời cho nhiều kiện hàng chưa hoàn tất nhận

3.7.1. Thiết kế cơ chế xác thực OTP

OTP (One-Time Password) là mã xác thực dùng một lần, được sử dụng trong SmartLocker để xác minh quyền nhận hàng trước khi Backend chấp nhận yêu cầu mở tủ.

Trong hệ thống, OTP được tạo riêng cho từng kiện hàng và liên kết với thông tin đơn hàng trong cơ sở dữ liệu. Mỗi mã gồm sáu chữ số và có thời gian hiệu lực nhất định.

Khác với hình thức nhập OTP thủ công, ứng dụng SmartLocker được xây dựng theo hướng tự động hiển thị mã OTP cho Cư dân đã đăng nhập đúng tài khoản.

Khi người dùng truy cập danh sách kiện hàng, Backend xác định những đơn hàng thuộc tài khoản hiện tại và cung cấp thông tin OTP còn hiệu lực. Ứng dụng hiển thị mã dưới dạng sáu ô số, giúp người dùng dễ dàng theo dõi mà không cần ghi nhớ hoặc nhập lại.

Mặc dù mã được điền tự động trên giao diện, thao tác nhận hàng vẫn cần được xác thực tại Backend. Ứng dụng không tự quyết định tính hợp lệ của OTP hoặc trực tiếp điều khiển relay.

Cơ chế này giúp giảm thao tác nhập liệu trong quá trình nhận hàng, đồng thời duy trì bước kiểm tra quyền truy cập tại máy chủ.

3.7.2. Xây dựng thuật toán phân bổ ô tủ tự động

Khi Shipper tạo đơn, Backend thực hiện kiểm tra danh sách ô tủ và xác định ô phù hợp để lưu trữ kiện hàng.

Trạng thái sử dụng của ô tủ được xác định dựa trên các bản ghi trong bảng packages.

Những kiện hàng có trạng thái READY được xem là đang chiếm dụng ô tủ tương ứng. Vì vậy, Backend không lựa chọn ô tủ đã liên kết với một đơn hàng READY khác.

Quy trình phân bổ được tổ chức theo các bước sau:

Backend tiếp nhận yêu cầu tạo đơn từ Shipper và kiểm tra quyền truy cập.

Hệ thống xác minh tài khoản người nhận và kích thước kiện hàng.

Backend xác định danh sách ô tủ phù hợp với cấu hình kích thước.

Hệ thống kiểm tra những ô đang có đơn hàng trạng thái READY.

Backend lựa chọn ô tủ còn trống theo thứ tự phân bổ được cấu hình.

Nếu tìm được ô phù hợp, Backend tạo đơn hàng và lưu mã ô tủ vào SQLite.

Nếu không còn ô phù hợp, Backend từ chối yêu cầu tạo đơn và trả về thông báo tương ứng.

Bảng 3.8. Điều kiện xác định khả năng phân bổ ô tủ

Điều kiện	Kết quả
Ô tủ không có đơn hàng READY	Có thể được xem xét phân bổ
Ô tủ đang có đơn hàng READY	Không được phân bổ cho đơn hàng mới
Không còn ô tủ phù hợp	Trả lỗi, không tạo đơn mới
Đơn hàng hoàn tất và không còn READY	Ô tủ có thể được xem xét sử dụng lại

3.7.3. Phối hợp giữa OTP, phân bổ ô tủ và điều khiển phần cứng

Trong SmartLocker, cơ chế OTP và phân bổ ô tủ không hoạt động độc lập mà được liên kết với nhau thông qua thông tin đơn hàng.

Khi Shipper tạo đơn, Backend xác định người nhận, lựa chọn ô tủ và tạo OTP. Dữ liệu được lưu vào SQLite với trạng thái READY.

Khi Cư dân yêu cầu nhận hàng, Backend kiểm tra quyền sở hữu đơn hàng và xác minh OTP. Nếu thông tin hợp lệ, Backend sử dụng mã ô tủ đã lưu để tạo lệnh mở tương ứng cho ESP32.

Thiết bị ESP32 nhận lệnh thông qua Backend, xác định kênh relay theo mã ô tủ và thực hiện điều khiển.

Theo luồng hoàn tất được thiết kế, sau khi ESP32 xác nhận OPEN, Cư dân lấy hàng và xác nhận kết thúc để ứng dụng gửi yêu cầu CLOSE. Tuy nhiên, ResidentScreen hiện chưa thực hiện bước gửi CLOSE; thao tác này cần được bổ sung và kiểm thử cùng Backend, firmware.

Ở chế độ ESP32, Backend chỉ chuyển đơn hàng sang PICKED_UP, xóa OTP và giải phóng ô tủ sau khi nhận ACK thành công của lệnh CLOSE. ACK của OPEN chỉ xác nhận lệnh mở đã được thực thi, chưa xác nhận hoàn tất nhận hàng.

Nhờ đó, ba nhóm chức năng gồm xác thực OTP, quản lý ô tủ và điều khiển phần cứng được liên kết thông qua cùng một bản ghi đơn hàng.

CHƯƠNG 4. TÍCH HỢP AI OCR VÀ PHẦN CỨNG IOT

Chương này trình bày hai thành phần nằm ở hai đầu luồng nghiệp vụ của SmartLocker: mô-đun **AI OCR** chạy trên ứng dụng Android của Shipper để trích xuất số điện thoại người nhận, và thiết bị **ESP32** điều khiển relay để đóng/mở ô tủ. Các hằng số và tham số nêu trong chương được lấy trực tiếp từ mã nguồn phiên bản ổn định của hệ thống.

4.1. Vai trò của AI trong nghiệp vụ Shipper

Mô-đun AI giải quyết một bài toán hẹp: **trích xuất số điện thoại (SĐT) người nhận** từ ảnh chụp bill giao hàng. Mô-đun không tham gia xác thực người dùng, cấp ô tủ hay điều khiển phần cứng.

SĐT người nhận là khóa nối giữa tờ bill vật lý và dữ liệu trong hệ thống: Backend dùng giá trị này (trường receiverPhone trong POST /packages) để tìm tài khoản RESIDENT và tạo kiện hàng. Việc chụp bill thay cho gõ tay 10 chữ số giúp Shipper giảm thời gian thao tác và hạn chế lỗi nhập sai.

Toàn bộ suy luận (inference) chạy cục bộ trên Android bằng ONNX Runtime; ảnh bill không được gửi lên server và request tạo đơn chỉ chứa receiverPhone, size và sender. Kết quả OCR chỉ được điền vào ô nhập liệu, Shipper vẫn giữ quyền quyết định cuối cùng và có thể sửa hoặc xóa SĐT trước khi xác nhận. Khi AI không đủ bằng chứng, hệ thống không trả SĐT và yêu cầu nhập tay thay vì tự chọn một số.

4.2. Pipeline nhận dạng số điện thoại

Pipeline được chia thành hai tầng. Tầng **React Native (TypeScript)** điều phối luồng và thực hiện hậu xử lý; tầng **native Android (Kotlin)** thực hiện xử lý ảnh và suy luận mô hình. Ví dụ trực quan của luồng được trình bày ở Hình 4.1. Thứ tự các bước như sau:

Bước 1 – Chụp ảnh: ShipperScreen dùng camera sau (react-native-vision-camera) chụp ảnh bill và lưu thành tệp JPEG.

Bước 2 – Resize ảnh: ảnh được thu nhỏ theo chế độ contain, giới hạn 1024 × 1440 pixel, chất lượng JPEG 90, chỉ thu nhỏ, không phóng to (react-native-image-resizer).

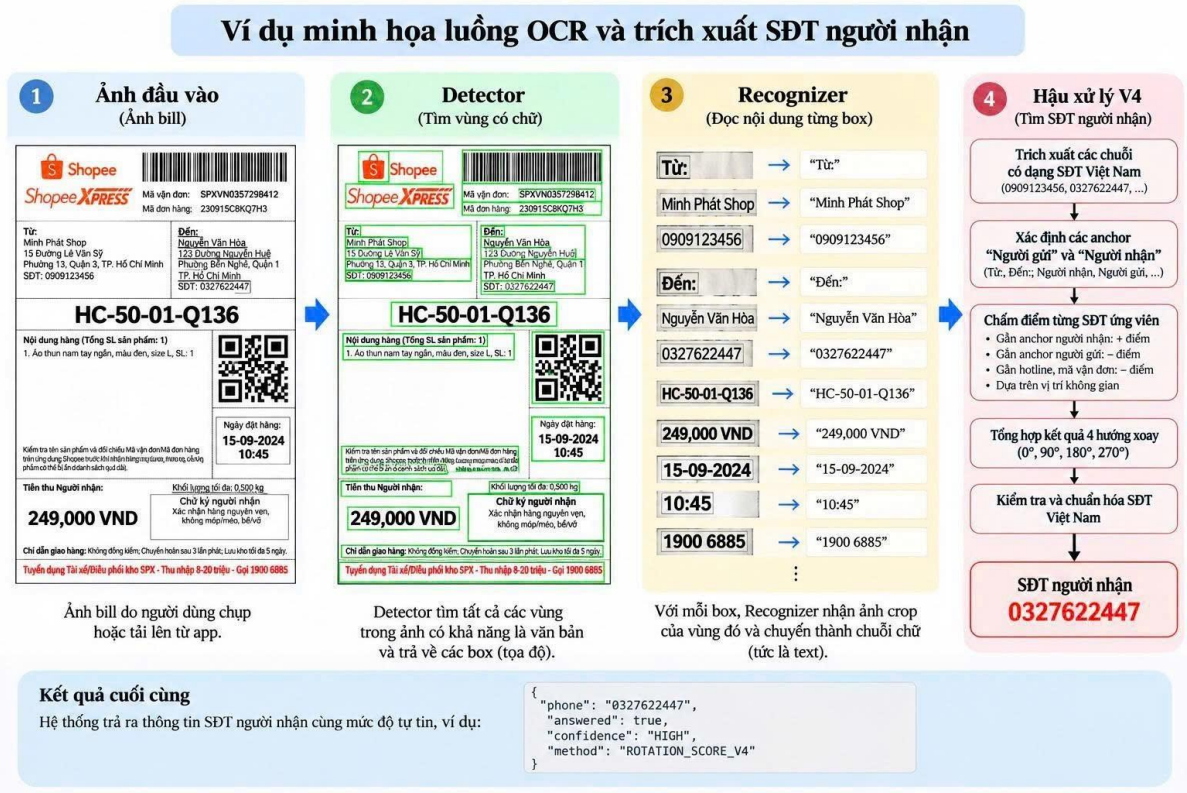
Bước 3 – Gọi native module: đường dẫn ảnh được truyền xuống native module SmartLockerOcrNative qua phương thức scanOrientation(path, degree).

Bước 4 – Tiền xử lý và hiệu chỉnh độ nghiêng: native module giải mã ảnh, xoay theo thẻ EXIF, thu nhỏ về tối đa 1024 × 1440, sau đó chạy thuật toán auto-deskew để sửa góc nghiêng nhỏ trong mặt phẳng ảnh.

Bước 5 – Detector: mô hình phát hiện vùng chữ trả về bản đồ xác suất; hậu xử lý tạo danh sách hộp bao (bounding box) cho từng vùng văn bản.

Bước 6 – Recognizer: từng vùng chữ được cắt, chuẩn hóa kích thước và nhận dạng theo lô (batch); giải mã CTC tạo chuỗi ký tự và độ tin cậy.

Bước 7 – Trích xuất SĐT: tầng TypeScript lọc chuỗi có dạng SĐT di động Việt Nam, xác định anchor người nhận/người gửi, chấm điểm ứng viên và tổng hợp kết quả qua các hướng xoay 0°, 90°, 270°, 180°.



Hình 4.1. Ví dụ minh họa luồng OCR và trích xuất SĐT người nhận

Hình 4.1 gồm bốn giai đoạn: ảnh bill đầu vào; Detector khoanh vùng các dòng chữ; Recognizer chuyển từng vùng thành văn bản; bước hậu xử lý V4 phân biệt số 0909123456 ở vùng người gửi với **0327622447** ở vùng người nhận và chọn số người nhận. Khung JSON cuối hình mang tính minh họa: kết quả thực tế của scanJpeg gồm phone, score, margin, confidence, method, orientation, lines, selectedLine, rotations và debug, không có trường answered.

4.2.1. Tiền xử lý ảnh trên native

Các thao tác tiền xử lý thực hiện trong SmartLockerOcrNativeModule.kt được tổng hợp ở Bảng 4.1.

Bảng 4.1. Các bước tiền xử lý ảnh trong native module

Bước	Cách thực hiện trong mã nguồn
Giải mã ảnh	BitmapFactory với inSampleSize lũy thừa của 2, định dạng ARGB_8888
Chuẩn hướng EXIF	Đọc TAG_ORIENTATION bằng ExifInterface, áp dụng phép xoay/lật tương ứng
Thu nhỏ	Giữ tỉ lệ, cạnh tối đa 1024 × 1440 pixel
Auto-deskew	Tìm góc trong khoảng −35° đến +35°: dò thô bước 5°, dò tinh ±4° quanh góc tốt nhất với bước 1°, trên ảnh mẫu có cạnh dài tối đa 448 pixel
Bộ nhớ đệm	Ảnh đã deskew được giữ lại theo đường dẫn; các lần gọi ở hướng xoay khác không phải giải mã lại

Thuật toán auto-deskew dùng phương pháp chiếu ngang (horizontal projection profile). Với mỗi góc thử θ, ảnh mẫu được xoay một góc θ; trên mỗi hàng pixel y, hệ thống đếm số điểm tối $n_y(\theta)$, tức số pixel có độ sáng $L = \frac{77R+150G+29B}{256}$ nhỏ hơn 185. Dải biên 4% ở mỗi phía được bỏ qua để tránh vùng trống do phép xoay tạo ra. Độ sắc nét của phân bố chiếu được tính theo công thức (4.1):

$$S(\theta) = \frac{1}{W_s H_s} \sum_y \left(n_y(\theta) - n_{y-1}(\theta) \right)^2 \tag{4.1}$$

trong đó W_s và H_s là bề rộng và chiều cao của vùng được xét. Khi các dòng chữ nằm ngang, hàng chữ và hàng trống tách bạch nên $S(\theta)$ đạt giá trị lớn nhất. Gọi θ^* là góc cho S lớn nhất; ảnh chỉ được xoay khi thỏa mãn đồng thời hai điều kiện:

$$|\theta^*| \geq 3° \text{ và } \frac{S(\theta^*)}{S(0)} \geq 1{,}65 \tag{4.2}$$

Điều kiện này tránh xoay sai với ảnh vốn đã thẳng. Các hướng xoay lớn 90°, 180°, 270° không do deskew xử lý mà do vòng lặp hướng xoay ở tầng TypeScript (mục 4.4.4).

4.3. Mô hình PP-OCRv6 và triển khai ONNX trên Android

4.3.1. Mô hình và dữ liệu huấn luyện

Hệ thống dùng bộ mô hình PP-OCRv6 của PaddleOCR ở cấu hình small [6]. Notebook huấn luyện dùng tệp cấu hình PP-OCRv6_small_det.yml, PP-OCRv6_small_rec.yml cùng trọng số pretrained chính thức. Detector (PP-OCRv6_small_det) giữ nguyên trọng số pretrained, không huấn luyện lại. Recognizer (PP-OCRv6_small_rec) được fine-tune qua hai giai đoạn: giai đoạn A trên bộ dữ liệu công khai MC-OCR 2021 gồm ảnh hóa đơn tiếng Việt chụp bằng điện thoại [9]; giai đoạn B fine-tune nhẹ trên MC-OCR kết hợp dữ liệu tăng cường dạng số điện thoại (phone augmentation). Cả hai mô hình được xuất sang định dạng ONNX với opset 11 để triển khai.

Bảng 4.2. Các tệp mô hình và từ điển triển khai trên Android

Tệp	Vai trò
model1_detector_V6.onnx	Detector phát hiện vùng chữ. Đóng gói vào assets với tên smartlocker/model1_detector.onnx
model2_recognizer_V6_TopK_F32.onnx	Recognizer nhận dạng ký tự, đầu ra rút gọn thành Top-2 dạng Float32. Đóng gói với tên smartlocker/model2_recognizer.onnx
ppocr_keys.txt	Từ điển ký tự, mỗi dòng một ký tự, dùng để giải mã chỉ số đầu ra của Recognizer

Phiên bản **TopK-F32** của Recognizer có hai đầu ra smartlocker_top2_values và smartlocker_top2_indices_f32 kích thước [N, T, 2], tức chỉ giữ hai ký tự có điểm cao nhất tại mỗi bước thời gian thay vì toàn bộ phân phối trên từ điển. Đầu ra nhỏ giúp giảm chi phí sao chép tensor và giải mã. Khi nạp mô hình, native module kiểm tra sự tồn tại của hai đầu ra này và báo lỗi nếu mô hình không đúng phiên bản. Tài liệu gói AI ghi nhận các thử nghiệm lượng tử hóa INT8 và Progressive ROI không đạt tiêu chí chất lượng nên không nằm trong bản ổn định.

4.3.2. Triển khai bằng ONNX Runtime

Ứng dụng khai báo thư viện com.microsoft.onnxruntime:onnxruntime-android phiên bản **1.29.0**, là gói ONNX Runtime chính thức cho Android [7]. Gradle được cấu hình noCompress cho đuôi .onnx để tệp mô hình không bị nén trong APK. Mã suy luận

viết bằng **Kotlin**, dùng các lớp OrtEnvironment, OrtSession và OrtSession.SessionOptions của API Java [7].

Bảng 4.3. Cấu hình suy luận của Detector và Recognizer

Tham số	Detector	Recognizer
Kích thước đầu vào	[1, 3, H′, W′], cạnh dài tối đa 736, H′ và W′ làm tròn về bội số của 32 theo công thức (4.3)	[N, 3, 48, 320], N tối đa 16 vùng chữ mỗi lô
Chuẩn hóa pixel	Thứ tự kênh BGR, chuẩn hóa theo công thức (4.4)	Thứ tự kênh BGR, chuẩn hóa theo công thức (4.5), phần đệm bên phải bằng 0
Số luồng CPU (intra-op)	4	6
Tối ưu đồ thị	ALL_OPT, chế độ thực thi SEQUENTIAL, bật CPU arena allocator	ALL_OPT, chế độ thực thi SEQUENTIAL, bật CPU arena allocator

Trước khi đưa vào Detector, ảnh kích thước $W \times H$ được thu nhỏ theo hệ số s (chỉ thu nhỏ, không phóng to) và làm tròn mỗi cạnh về bội số gần nhất của 32, tối thiểu 32 pixel:

$$s = \min\left(1;\ \frac{736}{\max(W;\ H)}\right)$$
$$W' = max\left(32;\ 32 \cdot \text{round}\left(\frac{sW}{32}\right)\right),\ \ H' = max\left(32;\ 32 \cdot \text{round}\left(\frac{sH}{32}\right)\right) \quad (4.3)$$

Giá trị mỗi kênh màu $x_c \in [0;\ 255]$, với $c \in \{B, G, R\}$, được chuẩn hóa theo công thức (4.4) đối với Detector và (4.5) đối với Recognizer:

$$\hat{x}_c = \frac{x_c/255 - \mu_c}{\sigma_c} \quad (4.4)$$

$$\hat{x}_c = \frac{x_c}{127{,}5} - 1 \quad (4.5)$$

trong đó $\mu = (0{,}485;\ 0{,}456;\ 0{,}406)$ và $\sigma = (0{,}229;\ 0{,}224;\ 0{,}225)$ là vectơ trung bình và độ lệch chuẩn, áp dụng lần lượt cho các kênh B, G, R. Với Recognizer, mỗi vùng chữ được co giãn về chiều cao 48 pixel, chiều rộng theo tỉ lệ ảnh nhưng không vượt quá 320 pixel; phần còn thiếu bên phải được đệm bằng 0.

Hậu xử lý Detector được triển khai tối giản: các pixel có xác suất từ 0,20 trở lên được gom thành thành phần liên thông theo 8 lân cận; thành phần quá nhỏ hoặc có xác

suất trung bình dưới 0,45 bị loại; hộp bao chữ nhật của mỗi thành phần được nới rộng 1,4 lần quanh tâm (unclip) và quy đổi về tọa độ ảnh. Các hộp được sắp xếp từ trên xuống, trái sang phải và giới hạn tối đa 80 hộp.

Giải mã Recognizer dùng CTC tham lam (greedy) [10]: tại mỗi bước thời gian lấy chỉ số có điểm cao nhất, gộp các nhãn lặp liền kề trên đường đi dự đoán rồi loại ký tự trống (blank, chỉ số 0); chỉ số $k \geq 1$ ánh xạ tới dòng thứ k trong ppocr_keys.txt, chỉ số kế tiếp sau từ điển được hiểu là dấu cách. Độ tin cậy của một dòng được tính theo công thức (4.6):

$$C = \frac{1}{K}\sum_{k=1}^{K}\sigma\left(v_k^{(1)} - v_k^{(2)}\right), \ \ \sigma(z) = \frac{1}{1+e^{-z}} \tag{4.6}$$

trong đó K là số ký tự được giữ lại sau giải mã; $v_k^{(1)}$ và $v_k^{(2)}$ là điểm Top-1 và Top-2 tại bước thời gian sinh ra ký tự thứ k. Trước khi cắt vùng chữ, hộp bao được nới thêm 8% bề rộng theo chiều ngang và 18% chiều cao theo chiều dọc về mỗi phía để tránh mất nét chữ ở biên.

Toàn bộ suy luận chạy trên một luồng nền riêng (single-thread executor) để không chặn luồng giao diện; không có lời gọi mạng nào trong pipeline OCR.

4.3.3. Bộ dữ liệu kiểm thử

Gói AI có hai bộ ảnh dùng để kiểm thử pipeline. Bộ robustness gồm 36 ảnh tạo từ 4 bill gốc (Shopee Express, Lazada Express, TikTok Shop/J&T); mỗi bill có 9 biến thể gồm ảnh gốc, xoay ngang 90° theo hai chiều, nghiêng khoảng +15°, chụp xiên trái, chụp xiên phải, chỉ chụp khối người nhận, chụp phần đầu có cả người gửi và người nhận, và chụp góc trên bên phải. Các biến thể chỉ thay đổi hình học hoặc vùng cắt, không thay đổi chữ số. Bộ unseen gồm 6 nhãn giao hàng có bố cục khác bộ robustness; notebook kiểm thử tự tạo 4 hướng xoay 0°, 90°, 180°, 270° cho mỗi ảnh.

Mỗi ảnh trong hai bộ dữ liệu đi kèm SĐT người nhận đúng để đối chiếu với kết quả của pipeline.

4.4. Hậu xử lý và lựa chọn số điện thoại người nhận

Một bill thường chứa nhiều chuỗi số: SĐT người gửi, SĐT người nhận, hotline, mã vận đơn, mã đơn hàng. Bước hậu xử lý (phiên bản V4) chọn SĐT người nhận dựa trên nội dung văn bản và vị trí không gian của các dòng OCR. Trước khi xử lý, các dòng có độ tin cậy dưới 0,2 bị loại và các dòng còn lại được sắp xếp theo thứ tự đọc.

4.4.1. Tìm chuỗi ứng viên bằng Regex

Chuỗi ứng viên được tìm trên từng dòng OCR bằng biểu thức chính quy (regular expression). Biểu thức nhận các chuỗi bắt đầu bằng 0, 84 hoặc +84, tiếp theo là một chữ số thuộc tập {3, 5, 7, 8, 9} và thêm 8 chữ số; giữa các chữ số được phép xen dấu cách, dấu chấm, gạch ngang hoặc dấu ngoặc. Một chuỗi khớp bị loại nếu ký tự liền trước hoặc liền sau nó là chữ cái hay chữ số, nhằm tránh cắt nhầm một đoạn của mã vận đơn dài. Với mỗi chuỗi khớp, hệ thống loại bỏ các ký tự không phải số để được chuỗi chữ số u (độ dài $|u|$), sau đó quy đổi về dạng bắt đầu bằng 0 theo công thức (4.7):

$$N(u) = \begin{cases} 0u_3u_4 \dots u_{11}, & \text{nếu } |u| = 11 \text{ và } u_1u_2 = 84 \\ u_3u_4 \dots u_{12}, & \text{nếu } |u| = 12 \text{ và } u_1u_2u_3 = 840 \\ u, & \text{trường hợp còn lại} \end{cases} \quad (4.7)$$

trong đó u_i là chữ số thứ i của u. Kết quả $p = N(u)$ chỉ được giữ làm ứng viên khi thuộc tập số di động hợp lệ P theo công thức (4.8):

$$P = \{0p_2p_3 \dots p_{10} \mid p_2 \in \{3,5,7,8,9\};\ \ p_3, \dots, p_{10} \in \{0, \dots, 9\}\} \quad (4.8)$$

tức SĐT di động 10 chữ số với đầu số 03, 05, 07, 08, 09.

Các dòng chứa từ khóa tiêu cực như hotline, cskh, tổng đài, mã vận đơn, mã đơn hàng, tracking, order id không sinh ứng viên, vì số xuất hiện trên những dòng này thường là số tổng đài hoặc mã tra cứu.

4.4.2. Xác định anchor

Anchor là dòng văn bản cho biết vai trò của khối thông tin xung quanh. Văn bản được chuẩn hóa về chữ thường, bỏ dấu tiếng Việt và ký tự đặc biệt, sau đó so khớp với hai nhóm từ khóa. Nhóm anchor người nhận gồm "người nhận", "thông tin người nhận", "receiver", "recipient", "consignee" và các dạng ngắn "đến", "to". Nhóm anchor người gửi gồm "người gửi", "thông tin người gửi", "sender", "ship from", "from" và dạng ngắn "từ".

Để chịu được lỗi OCR, mức khớp giữa dòng văn bản a và từ khóa b được đo bằng độ tương đồng dựa trên khoảng cách Levenshtein [12]:

$$\text{sim}(a;\ b) = 1 - \frac{\text{lev}(a;\ b)}{\max(|a|;\ |b|)} \quad (4.9)$$

trong đó $\text{lev}(a;\ b)$ là số phép chèn, xóa, thay ký tự tối thiểu để biến a thành b; $|a|$ và $|b|$ là độ dài hai chuỗi sau chuẩn hóa. Một dòng được gán vai trò khi độ tương

đồng với nhóm đó đạt từ 0,68 trở lên và lớn hơn độ tương đồng với nhóm còn lại ít nhất 0,08. Hệ thống còn giữ danh sách các dạng sai thường gặp khi OCR mất dấu (ví dụ "nguinhan", "nguoigi", "dn") để nhận diện trực tiếp.

4.4.3. Chấm điểm ứng viên

Với mỗi ứng viên, hệ thống đo khoảng cách từ dòng chứa ứng viên tới anchor người nhận, anchor người gửi và dòng tiêu cực gần nhất. Khoảng cách giữa dòng i và dòng j là khoảng cách Euclid giữa tâm hai hộp bao sau khi chuẩn hóa theo độ trải của bố cục:

$$d_{ij} = \sqrt{\left(\frac{x_i - x_j}{\Delta x}\right)^2 + \left(\frac{y_i - y_j}{\Delta y}\right)^2} \tag{4.10}$$

trong đó $(x_i;\ y_i)$ là tọa độ tâm hộp bao của dòng i; Δx và Δy là hiệu giữa giá trị lớn nhất và nhỏ nhất của hoành độ, tung độ tâm trên toàn bộ các dòng OCR (tối thiểu 1 pixel). Mức độ gần được quy đổi thành hệ số ảnh hưởng:

$$\mathrm{str}(d;\ r) = \begin{cases} 1 - \dfrac{d}{r}, & \text{nếu } d < r \\ 0, & \text{nếu } d \geq r \end{cases} \tag{4.11}$$

với r là bán kính ảnh hưởng. Ký hiệu d_R, d_S, d_N lần lượt là khoảng cách (4.10) tới anchor người nhận, anchor người gửi và dòng tiêu cực gần nhất; g_R, g_S là số dòng cách anchor người nhận, người gửi gần nhất theo thứ tự đọc. Các thành phần điểm được cộng dồn theo Bảng 4.4.

Bảng 4.4. Quy tắc chấm điểm ứng viên SĐT

Tiêu chí	Điểm
Gần anchor người nhận ($r = 0{,}55$)	$+12 \cdot \mathrm{str}(d_R;\ 0{,}55)$
Gần anchor người gửi ($r = 0{,}55$)	$-11 \cdot \mathrm{str}(d_S;\ 0{,}55)$
Gần dòng tiêu cực như hotline, mã vận đơn ($r = 0{,}32$)	$-8 \cdot \mathrm{str}(d_N;\ 0{,}32)$
Cách anchor người nhận không quá 4 dòng theo thứ tự đọc	$+2{,}5 \cdot (5 - g_R)/5$
Cách anchor người gửi không quá 4 dòng theo thứ tự đọc	$-2 \cdot (5 - g_S)/5$
Bill không có anchor người nhận nhưng có anchor người gửi	$+\min(3;\ 4d_S)$

Tiêu chí	Điểm
Dòng chứa nhãn "sđt", "phone", "tel", "mobile"	+0,75

Sau khi chấm điểm, kết quả trong một hướng xoay được quyết định theo Bảng 4.5.

Bảng 4.5. Quy tắc quyết định kết quả trong một hướng xoay

Trường hợp	Kết quả
Không có ứng viên	Không chọn SĐT, mức LOW
Một ứng viên	Chọn ứng viên; mức HIGH nếu điểm ≥ 4, ngược lại MEDIUM
Nhiều ứng viên, điểm cao nhất < 1 hoặc chênh lệch với ứng viên thứ hai < 1,25	Không chọn SĐT, mức LOW
Nhiều ứng viên, các trường hợp còn lại	Chọn ứng viên điểm cao nhất; mức HIGH nếu điểm ≥ 4 và chênh lệch ≥ 2, ngược lại MEDIUM

4.4.4. Xử lý nhiều hướng xoay

Do bill có thể bị chụp xoay ngang hoặc ngược, engine lần lượt thử các hướng 0°, 90°, 270°, 180°. Mỗi hướng gọi native scanOrientation một lần và dùng lại ảnh cơ sở đã deskew trong bộ nhớ đệm. Nếu hướng 0° cho kết quả HIGH với điểm từ 4 trở lên (fast path), vòng lặp dừng ngay mà không chạy ba hướng còn lại. Từ hướng thứ hai trở đi, sau mỗi hướng hệ thống tổng hợp kết quả tạm thời và dừng sớm nếu kết quả tổng hợp đạt HIGH đồng thời có ít nhất hai hướng cùng chọn một SĐT.

Khi tổng hợp, mỗi SĐT p xuất hiện ở các hướng đã chạy nhận điểm cuối:

$$F(p) = \sum_k \text{clip}(s_k(p);\ -12;\ 12) + \sum_k B_k(p) + 1{,}5 \cdot n^+(p) \qquad (4.12)$$

trong đó $s_k(p)$ là điểm của p trong hướng xoay k (bằng 0 nếu p không xuất hiện ở hướng đó), được giới hạn trong đoạn $[-12;\ 12]$; $B_k(p)$ là điểm thưởng bình chọn, bằng 4, 2 hoặc 0 khi hướng k chọn p với mức HIGH, MEDIUM hoặc LOW và bằng 0 nếu hướng đó không chọn p; $n^+(p)$ là số hướng mà p có điểm dương.

Hệ thống không trả SĐT nếu điểm cuối của SĐT đứng đầu không lớn hơn 0, hoặc chênh lệch điểm cuối giữa hai SĐT đứng đầu nhỏ hơn 2. Kết quả đạt mức HIGH khi

điểm cuối từ 6 trở lên và có ít nhất một hướng HIGH hoặc ít nhất hai hướng cùng chọn SĐT đó; các trường hợp còn lại là MEDIUM.

Kết quả trả về cho giao diện gồm phone (hoặc null), score, margin, confidence, method, orientation, danh sách dòng OCR, kết quả từng hướng và thông tin thời gian xử lý.



Hình 4.2. Bill chụp nghiêng (trái) và kết quả trên App Shipper khi phải chạy đủ bốn hướng xoay (phải)

Hình 4.2 là một lần chạy thử với bill chụp nghiêng. Hướng 0° chưa đạt điều kiện fast path nên engine chạy thêm 90°, 270°, 180° (rotations=4) và tổng hợp ra SĐT **0327622447** với method ROTATION_SCORE_V4, mức HIGH. Thời gian AI trong lần chạy này là 28 576 ms, do bốn lượt Detector và Recognizer được thực hiện tuần tự.

4.5. Tích hợp AI với ShipperScreen

4.5.1. Cầu nối React Native – native Android

Mô-đun AI được tích hợp theo cơ chế Native Module của React Native trên Android [1]. Ở phía native, lớp SmartLockerOcrNativeModule kế thừa ReactContextBaseJavaModule và đăng ký với tên SmartLockerOcrNative; các phương

thức được đánh dấu @ReactMethod và trả kết quả qua Promise. Lớp SmartLockerOcrNativePackage khai báo module này và MainApplication.kt thêm package vào danh sách package của ứng dụng.

Ở phía TypeScript, smartlockerOcr.ts truy cập module qua NativeModules.SmartLockerOcrNative, kiểm tra module đã được đăng ký và cung cấp hàm createSmartLockerOcr() trả về engine gồm hai hàm scanJpeg(path) và dispose(). Các phương thức native được tổng hợp ở Bảng 4.6.

Bảng 4.6. Các phương thức của SmartLockerOcrNativeModule

Phương thức	Chức năng
initialize()	Nạp hai mô hình ONNX và từ điển từ assets, tạo hai OrtSession; trả về số ký tự từ điển, kích thước lô và thời gian nạp
scanOrientation(path, degree)	Chạy Detector và Recognizer trên ảnh ở hướng 0/90/180/270; trả về danh sách dòng (tọa độ, văn bản, độ tin cậy) và thông số thời gian
dispose()	Đóng các session, giải phóng bitmap đệm và từ điển

4.5.2. Quản lý vòng đời engine

Engine được tạo một lần trong useEffect với mảng phụ thuộc rỗng, tức khi ShipperScreen được mount. Tham chiếu engine được giữ bằng useRef; giá trị của ref không đổi giữa các lần render và việc gán ref không gây render lại [11], do đó mô hình không bị nạp lại mỗi khi giao diện cập nhật. Hàm cleanup của useEffect gọi dispose() khi màn hình unmount; nếu màn hình đã unmount trong lúc mô hình đang nạp, engine vừa tạo được giải phóng ngay.

Các trạng thái isModelLoading, modelError và isScanning được dùng để khóa nút chụp và hiển thị lỗi; thao tác quét mới bị bỏ qua khi lượt quét trước chưa hoàn tất.

4.5.3. Luồng thao tác trên ShipperScreen

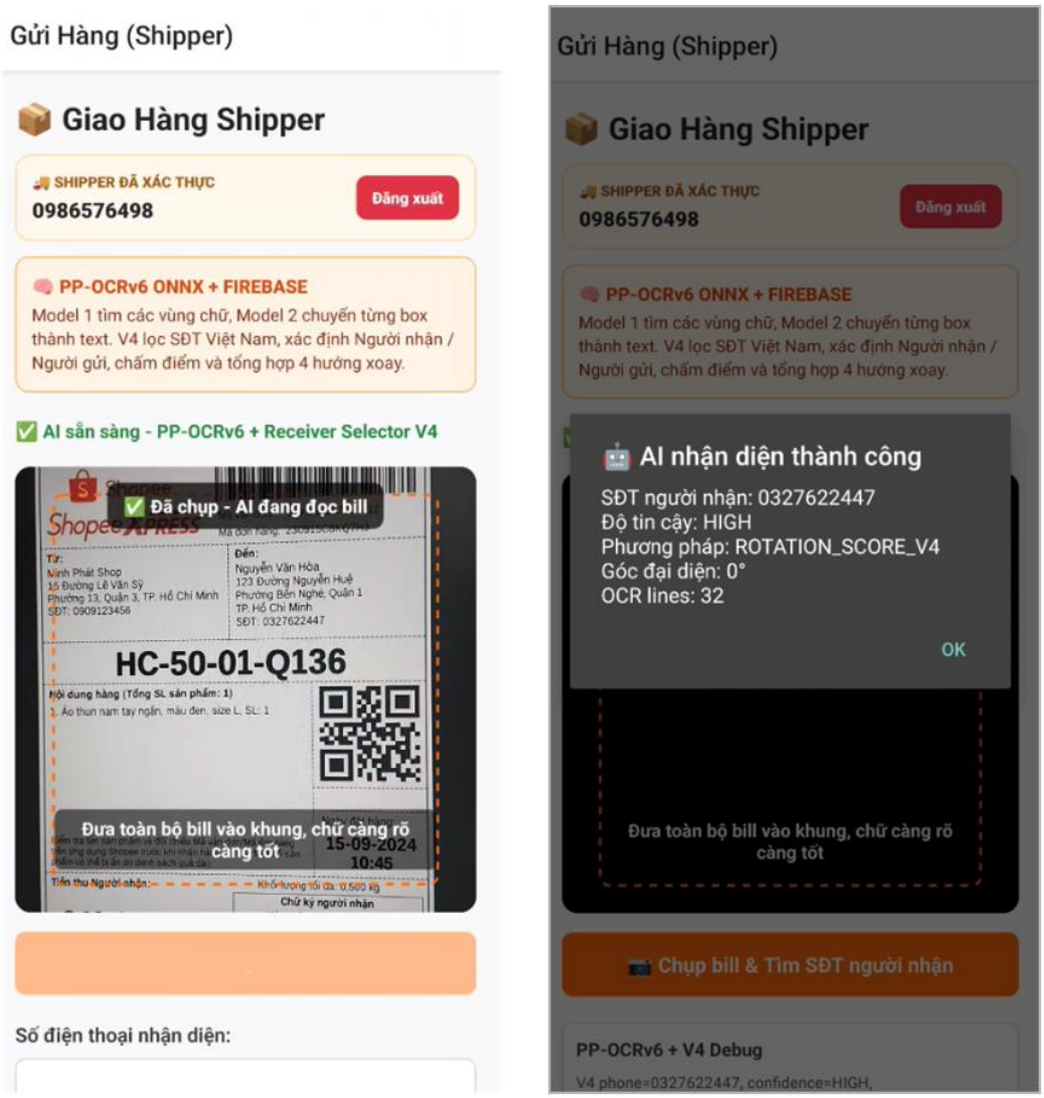
Bước 1: Shipper nhấn chụp; ứng dụng kiểm tra camera và engine đã sẵn sàng.

Bước 2: ảnh được chụp, resize và truyền vào engine.scanJpeg().

Bước 3: nếu có SĐT, giá trị được đưa vào TextInput (bàn phím số, tối đa 10 ký tự) và hiển thị thông báo gồm SĐT, mức tin cậy, phương pháp, góc xoay, thời gian xử

lý. Nếu không có SĐT, ứng dụng thông báo AI chưa đủ chắc chắn và đề nghị chụp lại hoặc nhập tay.

Bước 4: Shipper kiểm tra hoặc sửa SĐT. Khi tạo đơn, ứng dụng kiểm tra lại SĐT theo tập P ở công thức (4.8), sau đó gửi **POST /packages** kèm JWT (Authorization: Bearer) với các trường receiverPhone, size, sender.



Hình 4.3. Màn hình App Shipper khi chụp bill (trái) và hộp thoại kết quả nhận diện (phải)

Trong Hình 4.3, ảnh bên trái là thời điểm Shipper vừa bấm chụp: khung camera chứa toàn bộ bill và ứng dụng hiển thị trạng thái AI đang đọc bill. Ảnh bên phải là hộp thoại "AI nhận diện thành công" với SĐT người nhận **0327622447**, độ tin cậy HIGH, phương pháp ROTATION_SCORE_V4, góc đại diện 0° và 32 dòng OCR. Sau khi

Shipper bấm OK, SĐT đã nằm trong ô "Số điện thoại nhận diện" để kiểm tra trước khi tạo đơn.

4.6. Thiết kế phần cứng ESP32 và relay

Phần cứng gồm bo mạch **ESP32 DevKit V1** (module ESP32-WROOM-32) và **module relay 4 kênh**, mỗi kênh điều khiển cơ cấu khóa của một ô tủ. Firmware SmartLockerESP32.ino viết trên nền Arduino-ESP32, dùng các thư viện WiFi, WiFiClientSecure và ArduinoJson. ESP32 hoạt động ở chế độ **Station**, tức kết nối vào một điểm truy cập Wi-Fi có sẵn như một client [8].

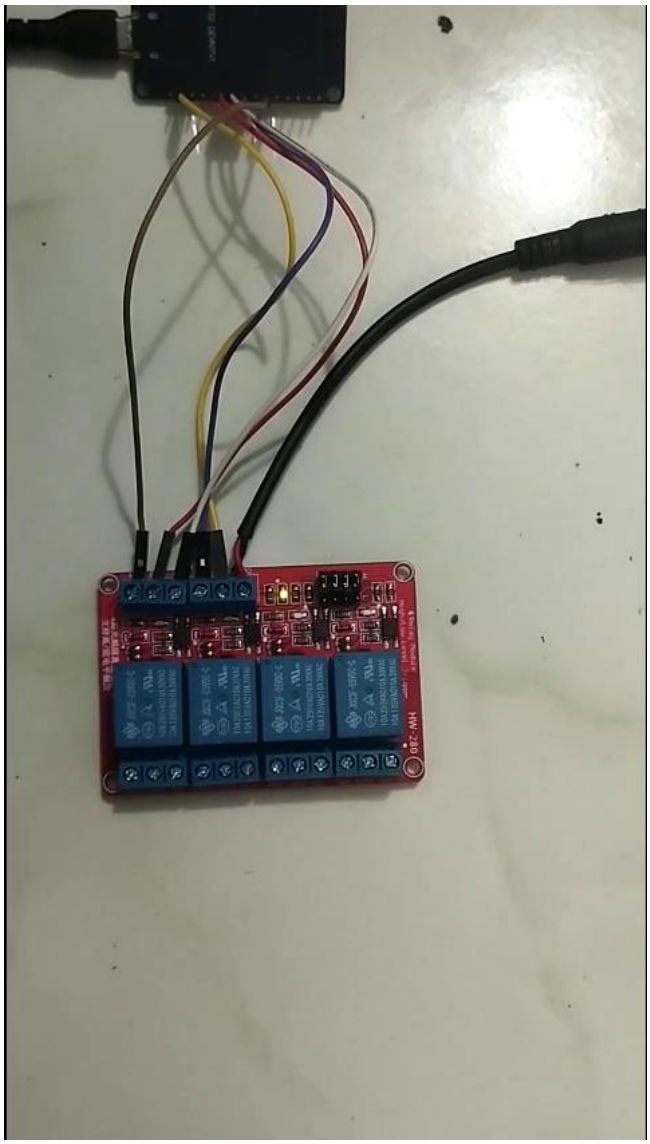
Bảng 4.7. Cấu hình hoạt động của firmware ESP32

Thông số	Giá trị trong firmware
Chế độ Wi-Fi	Station (WIFI_STA), tắt chế độ tiết kiệm năng lượng Wi-Fi (setSleep(false)), tự quản lý kết nối lại (setAutoReconnect(false))
Kết nối Backend	HTTPS, cổng 443, tới public domain của Backend trên Railway
Chu kỳ polling	1200 ms
Thời gian chờ kết nối Wi-Fi	25 giây; thử lại sau 10 giây nếu thất bại
Thời gian chờ HTTPS	15 giây; bộ đệm đọc phản hồi giới hạn khoảng 8192 byte
Mức điều khiển relay	HIGH = ON, LOW = OFF (jumper module relay đặt ở mức H)

Firmware được thiết kế theo nguyên tắc khởi động an toàn: trong setup(), hàm initRelays() ghi mức LOW lên từng chân trước khi đặt chân ở chế độ OUTPUT, sau đó tắt lại toàn bộ relay. Firmware không bật thử relay khi khởi động, nên ô tủ không tự mở khi mất điện rồi có điện trở lại. ESP32 không lưu dữ liệu nghiệp vụ như đơn hàng, SĐT hay OTP và không tự quyết định mở ô nào; thiết bị chỉ thực thi lệnh gồm id, action và locker do Backend cấp.

Trạng thái relay được giữ theo lệnh: lệnh OPEN bật relay của đúng ô và giữ ở mức ON, không tự tắt theo thời gian; relay chỉ tắt khi nhận lệnh CLOSE. Mỗi lệnh chỉ tác động lên relay của ô được chỉ định, các relay khác giữ nguyên. Về bảo mật, kết nối dùng TLS nhưng firmware gọi client.setInsecure(), tức không xác minh chứng chỉ máy

chủ; dữ liệu được mã hóa trên đường truyền nhưng thiết bị chưa chống được giả mạo máy chủ. Đây là giới hạn của phiên bản hiện tại.



Hình 4.4. Kết nối ESP32 với module relay 4 kênh

4.7. Ánh xạ ô tủ và GPIO

Mô hình phần cứng hiện tại dùng 4 ô tủ. Ánh xạ được khai báo trong mảng hằng LOCKERS[] của firmware và thống nhất với danh sách ALL_LOCKERS trong server.js của Backend (4 tủ A01, A02, B01, B05, cùng size M).

Bảng 4.8. Ánh xạ ô tủ – kênh relay – GPIO

Ô tủ	Kênh relay	GPIO ESP32	Mức điều khiển
A01	IN1	GPIO25	HIGH = ON (mở), LOW = OFF (đóng)

Ô tủ	Kênh relay	GPIO ESP32	Mức điều khiển
A02	IN2	GPIO26	HIGH = ON (mở), LOW = OFF (đóng)
B01	IN3	GPIO27	HIGH = ON (mở), LOW = OFF (đóng)
B05	IN4	GPIO32	HIGH = ON (mở), LOW = OFF (đóng)

Khi nhận lệnh, hàm findRelayPin() so khớp mã ô tủ với bảng trên để lấy chân GPIO. Mã ô tủ không có trong bảng được xem là lỗi: firmware không tác động relay và gửi ACK thất bại với thông điệp ô tủ chưa có relay vật lý.

4.8. Firmware và giao tiếp với Backend

4.8.1. Mô hình polling qua HTTPS

ESP32 chủ động hỏi Backend theo chu kỳ 1,2 giây bằng GET /hardware/commands/next. Cách tiếp cận này phù hợp khi ESP32 nằm sau NAT và không yêu cầu Backend mở kết nối trực tiếp tới thiết bị hoặc bổ sung message broker. Trong điều kiện kết nối ổn định, chu kỳ polling quyết định thời gian chờ trước lần hỏi kế tiếp; độ trễ thực tế còn phụ thuộc mạng, xử lý Backend, hàng đợi và thời gian thực thi, ACK.

4.8.2. Xác thực thiết bị

Mọi request từ ESP32 mang hai header X-Device-Key (khóa bí mật) và X-Device-Id (định danh thiết bị). Middleware hardwareAuthMiddleware của Backend so sánh X-Device-Key với biến môi trường ESP_DEVICE_KEY và trả HTTP 401 khi thiếu hoặc sai khóa. Giá trị X-Device-Id được ghi vào lệnh tại thời điểm thiết bị nhận lệnh và được dùng để kiểm tra đúng thiết bị khi ACK.

4.8.3. Vòng đời lệnh phần cứng

Lệnh được lưu trong bảng hardware_commands của Backend. Lệnh OPEN được tạo khi Cư dân xác minh OTP thành công (POST /packages/:packageId/redeem) hoặc khi Admin mở khẩn cấp (POST /admin/packages/:packageId/force-open); lệnh CLOSE được tạo qua POST /hardware/packages/:packageId/close. Trong ứng dụng hiện tại, App Cư dân chỉ gửi yêu cầu mở tủ và hiển thị hộp thoại "Mở tủ thành công" khi ESP32 ACK lệnh OPEN; lệnh CLOSE được App Admin gửi khi Admin bấm "OK - TẮT RELAY"

sau thao tác mở khẩn cấp. Sau ACK OPEN, kiện hàng vẫn ở trạng thái READY cho tới khi có ACK của lệnh CLOSE.

Bảng 4.9. Trạng thái của lệnh phần cứng

Trạng thái	Ý nghĩa
PENDING	Lệnh vừa được tạo, đang chờ thiết bị lấy
CLAIMED	Thiết bị đã lấy lệnh; Backend ghi device_id, claimed_at và tăng attempts
DONE	Thiết bị ACK thành công
FAILED	Thiết bị ACK thất bại; thông điệp lỗi được lưu vào error_message

Trình tự xử lý một lệnh:

Bước 1: ESP32 gửi GET /hardware/commands/next.

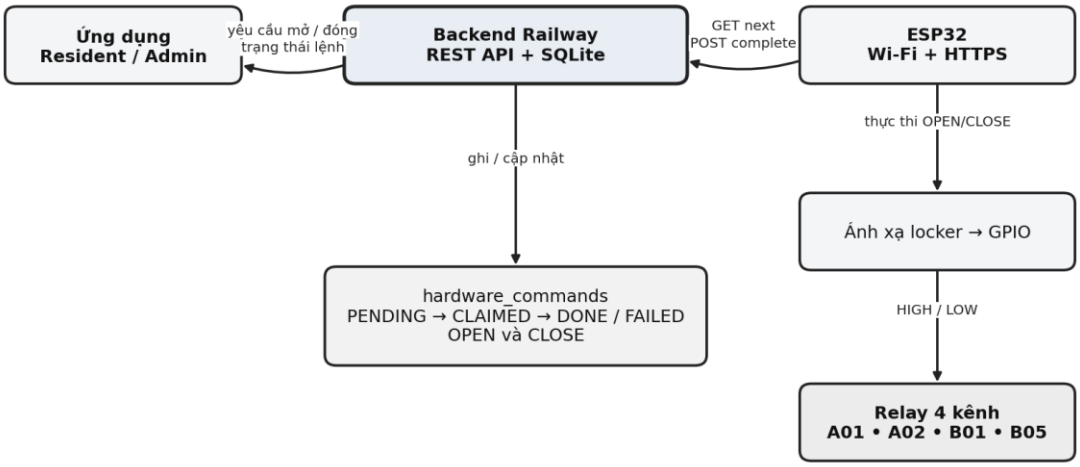
Bước 2: trong một transaction, Backend trả các lệnh CLAIMED quá hạn (mặc định 30 giây, cấu hình bằng HARDWARE_COMMAND_STALE_SECONDS) về PENDING, sau đó chọn **lệnh PENDING cũ nhất** (ORDER BY id ASC) và chuyển sang **CLAIMED**. Câu lệnh cập nhật có điều kiện status = 'PENDING' nên một lệnh không bị cấp cho hai lần polling.

Bước 3: Backend trả JSON gồm ok và command (id, action, locker…); command bằng null khi không có lệnh, khi đó ESP32 không thay đổi relay.

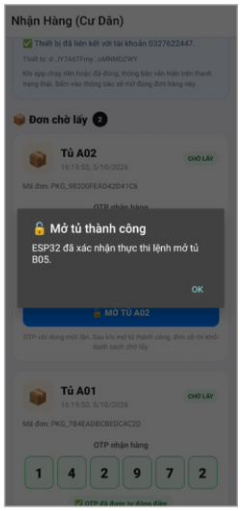
Bước 4: ESP32 kiểm tra id, action, locker; thực hiện OPEN (đặt chân HIGH) hoặc CLOSE (đặt chân LOW). Action khác OPEN/CLOSE được xem là lỗi.

Bước 5: ESP32 gửi **POST /hardware/commands/:commandId/complete** với body JSON gồm success và message để xác nhận (ACK).

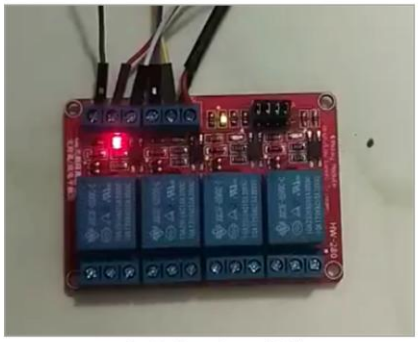
Bước 6: Backend kiểm tra lệnh tồn tại và thuộc đúng thiết bị (sai thiết bị trả HTTP 409). Với ACK OPEN thành công, lệnh chuyển DONE và kiện hàng vẫn ở trạng thái READY. Với ACK CLOSE thành công, kiện hàng chuyển PICKED_UP, dữ liệu OTP bị xóa và nhật ký hoạt động được ghi. ACK thất bại chuyển lệnh sang FAILED.



Hình 4.5. Sơ đồ giao tiếp Backend - ESP32 - relay



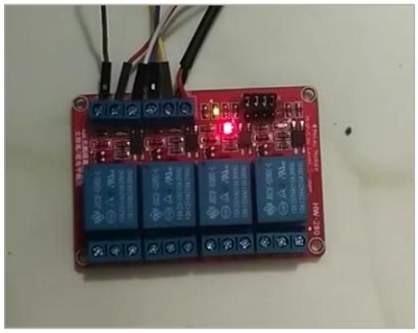
(a) Mở tủ B05



(b) Relay được kích



(c) Mở tủ A02



(d) Relay được kích

Hình 4.6. Mở tủ trên App Cư dân và trạng thái module relay: (a), (c) hộp thoại "Mở tủ thành công" của tủ B05 và A02; (b), (d) relay tương ứng được kích

Hình 4.6 ghi lại hai lần mở tủ liên tiếp trên App Cư dân. Sau khi Cư dân bấm "MỞ TỦ", ứng dụng hỏi trạng thái lệnh mỗi giây (tối đa 45 giây) và chỉ hiển thị hộp thoại "Mở tủ thành công – ESP32 đã xác nhận thực thi lệnh mở tủ …" khi lệnh OPEN chuyển sang DONE (a, c). Ở phía thiết bị, ESP32 đặt chân GPIO của ô tủ lên mức HIGH và đèn báo của kênh relay tương ứng trên module sáng (b, d). Theo Bảng 4.8, lệnh mở B05 tác động kênh IN4 (GPIO32), lệnh mở A02 tác động kênh IN2 (GPIO26). Relay giữ trạng thái ON cho tới khi ESP32 nhận lệnh CLOSE.

4.8.4. Xử lý mất kết nối

Khi mất Wi-Fi, firmware giữ nguyên trạng thái relay hiện tại, không tự đóng hay mở ô nào và ngừng polling. Firmware thử kết nối lại sau mỗi 10 giây; khi có mạng trở

lại, trạng thái relay vẫn không đổi và polling tiếp tục. Các request HTTPS lỗi (DNS, TLS, timeout) hoặc trả mã HTTP khác 200 chỉ được ghi log, firmware không tác động relay trong các trường hợp này.

Ở phía Backend, lệnh được tạo trong thời gian thiết bị mất kết nối vẫn ở trạng thái PENDING và được lấy theo thứ tự khi thiết bị kết nối lại. Lệnh đã CLAIMED nhưng chưa được ACK sẽ được trả về PENDING sau thời gian quá hạn.

4.8.5. Chuyển từ Software Mode sang ESP32 Mode

Backend chọn chế độ bằng biến môi trường HARDWARE_MODE, mặc định là SOFTWARE. Ở chế độ SOFTWARE, khi Cư dân xác minh OTP thành công, Backend chuyển kiện hàng sang PICKED_UP ngay mà không tạo lệnh phần cứng; chế độ này giữ tương thích với bản chạy khi chưa có phần cứng.

Ở chế độ ESP32, khi xác minh OTP thành công, Backend tạo lệnh OPEN và trả trạng thái WAITING_HARDWARE kèm commandId cho ứng dụng; kiện hàng chỉ chuyển PICKED_UP sau khi ESP32 ACK lệnh CLOSE. Nếu đặt HARDWARE_MODE=ESP32 nhưng thiếu ESP_DEVICE_KEY, Backend trả HTTP 503. Để chuyển sang chế độ này, cần đặt trên Railway các biến HARDWARE_MODE=ESP32, ESP_DEVICE_KEY, ESP_DEVICE_ID và tùy chọn HARDWARE_COMMAND_STALE_SECONDS; firmware dùng cùng ESP_DEVICE_KEY và ESP_DEVICE_ID khai báo trong tệp secrets.h.

4.9. Tích hợp AI, phần mềm và IoT

Luồng tích hợp AI OCR - Backend - ESP32 và các khóa dữ liệu liên kết được tổng hợp ở Bảng 4.10.

Bảng 4.10. Các khóa liên kết dữ liệu xuyên suốt hệ thống

Khóa	Phạm vi liên kết
receiverPhone	Nối kết quả OCR trên bill với tài khoản Cư dân trong Backend
public_id	Định danh kiện hàng dùng chung cho package, thông báo, nhật ký hoạt động và lệnh phần cứng (package_public_id)
locker	Nối dữ liệu kiện hàng trên Backend với kênh relay và chân GPIO trên ESP32
commandId	Nối lệnh do Backend tạo với ACK của ESP32 và trạng thái lệnh mà ứng dụng theo dõi

CHƯƠNG 5. THỰC NGHIỆM, KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Môi trường và phạm vi kiểm thử

Hệ thống được kiểm tra theo sáu nhóm thành phần: ứng dụng Android React Native, Backend Node.js/Express triển khai trên Railway, cơ sở dữ liệu SQLite, dịch vụ Firebase Cloud Messaging, mô-đun AI OCR chạy cục bộ bằng ONNX Runtime và mô hình ESP32 điều khiển relay. Phạm vi đánh giá tập trung vào luồng giao hàng, nhận hàng bằng OTP, phân quyền ba vai trò, thông báo, phân bổ ô tủ và vòng đời lệnh OPEN/CLOSE.

5.2. Kết quả kiểm thử tổng hợp

Bảng 5.1. Tổng hợp kết quả kiểm thử các thành phần chính

HẠNG MỤC	KẾT QUẢ CHÍNH	ĐÁNH GIÁ
Xác thực và phân quyền	Ba vai trò được kiểm tra ở Mobile App và Backend; Admin và Shipper dùng mã đăng ký riêng.	Đạt ở mức nguyên mẫu
AI OCR	Ảnh bill được xử lý cục bộ; hệ thống phát hiện vùng chữ, nhận dạng, chọn số điện thoại và cho phép Shipper sửa trước khi tạo đơn.	Đạt; phụ thuộc chất lượng ảnh
Tạo kiện và cấp ô	Backend tìm Cư dân, chọn ô còn trống, tạo package và trả lỗi 409 khi không còn ô phù hợp.	Đạt trong mô hình bốn ô
Thông báo FCM	Thông báo hỗ trợ foreground, background, cold start và điều hướng đến màn hình kiện hàng.	Đạt; phụ thuộc token và kết nối mạng
OTP nhận hàng	OTP sáu chữ số có thời hạn, gắn với đúng chủ sở hữu package và có chức năng cấp lại.	Đạt; cần bổ sung giới hạn số lần nhập

HẠNG MỤC	KẾT QUẢ CHÍNH	ĐÁNH GIÁ
ESP32 và relay	Thiết bị polling lệnh, ánh xạ ô tủ với GPIO, thực thi OPEN/CLOSE và gửi ACK về Backend.	Đạt ở mức mô hình
Mất kết nối	Lệnh đang chờ được giữ lại; lệnh CLAIMED quá hạn có thể trở về PENDING để xử lý lại.	Có cơ chế phục hồi cơ bản

5.3. Đánh giá kết quả tích hợp

Các thành phần đã được kết nối theo một luồng dữ liệu thống nhất: số điện thoại liên kết người nhận; package liên kết người dùng và ô tủ; OTP bảo vệ thao tác nhận hàng; hardware command liên kết yêu cầu trên ứng dụng với ESP32. Backend giữ vai trò nguồn trạng thái trung tâm và kiểm tra quyền truy cập trước khi thay đổi dữ liệu hoặc tạo lệnh phần cứng.

Kết quả cho thấy mô hình phù hợp để chứng minh khả năng tích hợp ứng dụng di động, AI OCR và IoT. Tuy nhiên, ACK của ESP32 mới xác nhận firmware đã ghi mức GPIO, chưa chứng minh trạng thái cửa vật lý. Vì vậy, báo cáo chỉ kết luận ở mức nguyên mẫu tích hợp và không mở rộng kết quả sang môi trường vận hành thực tế.

5.4. Hạn chế của hệ thống

- Luồng Cư dân ở chế độ ESP32 chưa gửi CLOSE sau OPEN; vì vậy chưa hoàn tất tự động việc tắt relay, cập nhật PICKED_UP và giải phóng ô tủ.
- Mô hình chỉ gồm bốn ô tủ kích thước M và chưa hỗ trợ nhiều cụm tủ hoặc nhiều vị trí.
- SQLite phù hợp với một Backend nguyên mẫu nhưng chưa tối ưu cho triển khai nhiều instance và tải đồng thời cao.
- Phần cứng chưa có cảm biến cửa; ACK chỉ phản ánh trạng thái lệnh tại vi điều khiển.
- Bộ dữ liệu đánh giá OCR còn nhỏ và chưa đại diện đầy đủ cho nhiều mẫu bill, điều kiện sáng và góc chụp.
- Hệ thống chưa có bộ kiểm thử tự động đầy đủ cho API, ứng dụng, migration và vòng đời lệnh phần cứng.

5.5. Hướng phát triển

- Hoàn thiện thao tác xác nhận lấy hàng và gửi CLOSE từ ResidentScreen; kiểm thử toàn bộ chuỗi OPEN, ACK, CLOSE và cập nhật trạng thái kiện hàng.
- Bổ sung cảm biến cửa, cơ chế tự tắt relay và đồng bộ trạng thái vật lý với trạng thái phần mềm.
- Bổ sung transaction, ràng buộc duy nhất, idempotency và rate limit cho các thao tác tạo kiện, OTP và điều khiển thiết bị.
- Mở rộng dữ liệu OCR, ghi nhận phiên bản mô hình và xây dựng bộ benchmark trên thiết bị Android.
- Chuẩn hóa lớp API dùng chung, bảo vệ thông tin xác thực trên thiết bị và bổ sung kiểm thử tự động trong quy trình phát triển.
- Khi mở rộng quy mô, chuyển sang hệ quản trị cơ sở dữ liệu phù hợp và xây dựng cơ chế quản lý nhiều cụm tủ.

KẾT LUẬN

Đồ án đã xây dựng một kiến trúc SmartLocker trong đó phần mềm đóng vai trò điều phối trung tâm. Mobile App React Native cung cấp luồng riêng cho Shipper, Cư dân và Quản trị viên; Backend Node.js/Express trên Railway xử lý xác thực, phân quyền, package, phân bổ locker, OTP, FCM, lịch sử và command; SQLite lưu dữ liệu bền vững; AI OCR PP-OCRv6/ONNX hỗ trợ Shipper nhận dạng số điện thoại; ESP32 nhận lệnh qua Backend và điều khiển bốn kênh relay.

Điểm nổi bật là sự liên kết xuyên suốt giữa số điện thoại, user, package, locker và hardware command. OTP và role được kiểm tra ở server, Shipper không nhận OTP, lệnh phần cứng có ACK, và Admin có thể theo dõi lịch sử. Gói AI stable được tích hợp ở mức native Android, có detector, recognizer, hậu xử lý ngữ nghĩa và fallback theo góc ảnh.

Qua rà soát mã nguồn, hệ thống phù hợp với mức prototype tích hợp. Tuy nhiên luồng Cư dân trong ESP32 mode còn thiếu thao tác CLOSE sau khi OPEN thành công. Đây là hạng mục bắt buộc hoàn thiện và kiểm thử trước khi kết luận package được nhận, locker được giải phóng và relay đã tắt. Việc nêu rõ giới hạn này giúp báo cáo phản ánh đúng trạng thái kỹ thuật hiện tại.

TÀI LIỆU THAM KHẢO

[1] React Native, Android Native Modules, https://reactnative.dev/docs/legacy/native-modules-android.

[2] Node.js, Node.js Documentation, https://nodejs.org/docs/latest/api/.

[3] Express.js, Express Web Framework Documentation, https://expressjs.com/.

[4] SQLite, SQLite Documentation, https://www.sqlite.org/docs.html.

[5] Google Firebase, Firebase Cloud Messaging Documentation, https://firebase.google.com/docs/cloud-messaging.

[6] PaddlePaddle, PP-OCRv6 Documentation, https://github.com/PaddlePaddle/PaddleOCR/blob/main/docs/version3.x/algorithm/PP-OCRv6/PP-OCRv6.en.md.

[7] Microsoft, ONNX Runtime Java and Android Documentation, https://onnxruntime.ai/docs/get-started/with-java.html.

[8] Espressif Systems, ESP-IDF Programming Guide for ESP32, https://docs.espressif.com/projects/esp-idf/en/latest/esp32/.

[9] RIVF2021 MC-OCR Organizers, Mobile-Captured Image Document Recognition for Vietnamese Receipts, 2021, https://www.rivf2021-mc-ocr.vietnlp.com/.

[10] A. Graves, S. Fernández, F. Gomez và J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," ICML, 2006, tr. 369–376, https://www.cs.toronto.edu/~graves/icml_2006.pdf.

[11] React, Built-in React Hooks, https://react.dev/reference/react/hooks.

[12] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Dokl. Akad. Nauk SSSR, tập 163, số 4, 1965, tr. 845–848, https://www.mathnet.ru/eng/dan31411.

LINK GITHUB : https://github.com/NgocThien12Yuna/smart-locker